Peering into the twilight - a systematic review on visual-based investigation of worldwide mesophotic ecosystems

Claudia Campanini

2026-06-18

Table of Contents

DATA PREPARATION

Load libraries

```
library(here)

## here() starts at C:/Users/claud/Documents/COMUNE/STUDIO_LAVORO/Data_analys
is/Projects/Mesophotic_systematic_lit_rev_analysis
```

```
library(dplyr)

##
## Attaching package: 'dplyr'

## The following objects are masked from 'package:stats':
##
##     filter, lag

## The following objects are masked from 'package:base':
##
##     intersect, setdiff, setequal, union

library(ggplot2)
library(readxl)   # to import excel data sets
library(magrittr)
library(stringr) #many data wrangling functions
library(readxl)
```

Load data

```
raw <- read_excel(here("Extracted_data_analysis","Input_data","extracted_raw.
xlsx"))
```

Cleaning

Clean year columns

```
#keep the year only in the year column (in some rows exact time is specified)
raw$publicationYear <-  stringr::str_remove(raw$publicationYear,
                                pattern = "\\-.*")
```

MISSING / ND / NOTAPP VALUES

Create a summary table

```
library(purrr)

##
## Attaching package: 'purrr'

## The following object is masked from 'package:magrittr':
##
##     set_names

library(tibble)


# Get all column names
cols <- colnames(raw)
  names(raw[,-c(1:13)])
```

```
##  [1] "meso_direct_indirect"          "meso_samplingPlatform"
##  [3] "taxonomic_publication"         "meso_substrate"
##  [5] "meso_habitat"                  "meso_protection"
##  [7] "taxonomicTargetLevel"          "habit"
##  [9] "environmentalPosition"         "detected_meso_seagrass"
## [11] "detected_meso_algae"           "detected_meso_cnidaria"
## [13] "detected_meso_sponges"         "detected_meso_echino"
## [15] "detected_meso_mollusca"        "detected_meso_chondrichthyes"
## [17] "detected_meso_fish"            "detected_meso_crustacea"
## [19] "detected_meso_other_taxa"      "indicators"
## [21] "collectedDataType"             "meso_protocol_campaign"
## [23] "samplingUnit"                  "visuallySurveyedSurface"
## [25] "meso_design"                   "meso_replicates"
## [27] "temporalReplicates"            "abioticVariablesCollection"
## [29] "abioticVariables"              "molecularAnalyses"
## [31] "targetTaxa"                    "DOI"
## [33] "meso_dimensions"               "observationalORexperimental"
## [35] "max_depth_direct"              "max_depth_hov"
## [37] "max_depth_rov"                 "min_depth_hov"
## [39] "min_depth_rov"                 "max_depth_ccr"
## [41] "max_depth_open"                "min_depth_ccr"
## [43] "min_depth_open"                "max_depth_scuba"
## [45] "min_depth_scuba"               "field_cons"
## [47] "field_commdesc"                "field_map"
## [49] "field_impass"                  "field_troph"
## [51] "field_taxo"                    "field_dist"
## [53] "field_aqua"                    "field_method"
## [55] "field_physio"                  "field_func"
## [57] "field_habtype"                 "field_fishery"
## [59] "field_stock"                   "field_paleo"
## [61] "field_phylo"                   "field_rest"
## [63] "field_archaeo"                 "field_micro"
## [65] "field_biomedicine"             "field_chem"
## [67] "field_Molecular_ecology"       "field_Acoustics"
## [69] "field_environmental_drivers"   "field_Behaviour"
## [71] "field_Geomorphology"           "field_Taxonomy"
```

```r
# Create a summary list
summary_list <- map(cols, function(col) {
  data <- raw[,-c(1:13)][[col]]

  tibble(
    Column     = col,
    unique     = paste(unique(data), collapse = ", "),
    nd         = sum(data == "nd", na.rm = TRUE),
    notapp     = sum(data == "notapp", na.rm = TRUE),
    assigned   = sum(!is.na(data) & data != "nd" & data != "notapp"),
    `NA`         = sum(is.na(data)),
    total      = sum(data == "nd", na.rm = TRUE) +
                 sum(data == "notapp", na.rm = TRUE) +
```

```
                sum(!is.na(data) & data != "nd" & data != "notapp")
  )
})

# Combine into one dataframe
summary_df <- bind_rows(summary_list)

summary_df
```

```
## # A tibble: 85 × 7
##    Column          unique    nd notapp assigned  `NA` total
##    <chr>           <chr>  <int>  <int>    <int> <int> <int>
##  1 File            ""         0      0        0     0     0
##  2 recordTitle     ""         0      0        0     0     0
##  3 recordAuthors   ""         0      0        0     0     0
##  4 publicationYear ""         0      0        0     0     0
##  5 province        ""         0      0        0     0     0
##  6 prov_code       ""         0      0        0     0     0
##  7 ecoreg          ""         0      0        0     0     0
##  8 scale           ""         0      0        0     0     0
##  9 ecoreg_code     ""         0      0        0     0     0
## 10 location        ""         0      0        0     0     0
## # ℹ 75 more rows
```

```
rm(summary_list, cols)
```

Create a summary table excluding purely taxonomic studies

```
summary_methods_df <- raw %>%
  filter(taxonomic_publication == "taxa++" |  is.na(taxonomic_publication) )

cols <- colnames(raw)
  names(raw[,-c(1:13)])
```

```
##  [1] "meso_direct_indirect"         "meso_samplingPlatform"
##  [3] "taxonomic_publication"        "meso_substrate"
##  [5] "meso_habitat"                 "meso_protection"
##  [7] "taxonomicTargetLevel"         "habit"
##  [9] "environmentalPosition"        "detected_meso_seagrass"
## [11] "detected_meso_algae"          "detected_meso_cnidaria"
## [13] "detected_meso_sponges"        "detected_meso_echino"
## [15] "detected_meso_mollusca"       "detected_meso_chondrichthyes"
## [17] "detected_meso_fish"           "detected_meso_crustacea"
## [19] "detected_meso_other_taxa"     "indicators"
## [21] "collectedDataType"            "meso_protocol_campaign"
## [23] "samplingUnit"                 "visuallySurveyedSurface"
## [25] "meso_design"                  "meso_replicates"
## [27] "temporalReplicates"           "abioticVariablesCollection"
## [29] "abioticVariables"             "molecularAnalyses"
## [31] "targetTaxa"                   "DOI"
```

```
## [33] "meso_dimensions"              "observationalORexperimental"
## [35] "max_depth_direct"             "max_depth_hov"
## [37] "max_depth_rov"                "min_depth_hov"
## [39] "min_depth_rov"                "max_depth_ccr"
## [41] "max_depth_open"               "min_depth_ccr"
## [43] "min_depth_open"               "max_depth_scuba"
## [45] "min_depth_scuba"              "field_cons"
## [47] "field_commdesc"               "field_map"
## [49] "field_impass"                 "field_troph"
## [51] "field_taxo"                   "field_dist"
## [53] "field_aqua"                   "field_method"
## [55] "field_physio"                 "field_func"
## [57] "field_habtype"                "field_fishery"
## [59] "field_stock"                  "field_paleo"
## [61] "field_phylo"                  "field_rest"
## [63] "field_archaeo"                "field_micro"
## [65] "field_biomedicine"            "field_chem"
## [67] "field_Molecular_ecology"      "field_Acoustics"
## [69] "field_environmental_drivers"  "field_Behaviour"
## [71] "field_Geomorphology"          "field_Taxonomy"

# Create a summary list
summary_methods_list <- map(cols, function(col) {
  data <- summary_methods_df[,-c(1:13)][[col]]

  tibble(
    Column    = col,
    unique    = paste(unique(data), collapse = ", "),
    nd        = sum(data == "nd", na.rm = TRUE),
    notapp    = sum(data == "notapp", na.rm = TRUE),
    assigned  = sum(!is.na(data) & data != "nd" & data != "notapp"),
    `NA`        = sum(is.na(data)),
    total     = sum(data == "nd", na.rm = TRUE) +
                sum(data == "notapp", na.rm = TRUE) +
                sum(!is.na(data) & data != "nd" & data != "notapp")
  )
})

# Combine into one dataframe
summary_methods_df <- bind_rows(summary_methods_list)

summary_methods_df

## # A tibble: 85 × 7
##    Column          unique    nd notapp assigned  `NA` total
##    <chr>           <chr>  <int>  <int>    <int> <int> <int>
##  1 File            ""         0      0        0     0     0
##  2 recordTitle     ""         0      0        0     0     0
##  3 recordAuthors   ""         0      0        0     0     0
##  4 publicationYear ""         0      0        0     0     0
```

```
##  5 province        ""          0      0        0     0     0
##  6 prov_code       ""          0      0        0     0     0
##  7 ecoreg          ""          0      0        0     0     0
##  8 scale           ""          0      0        0     0     0
##  9 ecoreg_code     ""          0      0        0     0     0
## 10 location        ""          0      0        0     0     0
## # ℹ 75 more rows

rm(summary_methods_df, cols)
```

GEOGRAPHICAL DISTRIBUTION

```
location <- raw[,c (
  "recordTitle","DOI","publicationYear","prov_code","ecoreg_code")]
```

Load Marine Ecoregion of the World data

```
meow <- read.csv(file=here("Extracted_data_analysis","Input_data","MEOW.csv"))
```

Provinces

```
prov <- location

# extract all unique province levels
prov_lvls <- unique(unlist(str_split(location$prov_code, "_")))

# create columns with exact matching
prov[, prov_lvls] <- t(sapply(location$prov_code, function(x) {
  parts <- str_split(x, "_")[[1]]
  as.integer(prov_lvls %in% parts)
}))

rm(prov_lvls)
```

Create a column with the sum of provinces per publication

```
prov %<>%
   mutate(
     prov_per_record = rowSums(
     across(
       where(is.numeric)
       )
     )
   )

prov$prov_per_record <- as.factor(prov$prov_per_record)

prov_per_record_df <- prov %>%
  group_by(prov_per_record) %>%
  summarise(n_records = length(prov_per_record)) %>%
```

```
  mutate(percentage= round(n_records / sum(n_records) * 100,0))

prov_per_record_df
```

```
## # A tibble: 5 × 3
##   prov_per_record n_records percentage
##   <fct>                <int>      <dbl>
## 1 1                      955         95
## 2 2                       40          4
## 3 3                        4          0
## 4 4                        1          0
## 5 10                       1          0
```

Create a data frame with number of publication per province

```
prov_tot <-
  prov [, -(length(prov)-1)] %>% # filter out the sum column
  summarise(across(where(is.numeric),
                   list(sum)
                   )
            )

prov_tot <- reshape2::melt(prov_tot, id.vars = NULL)

colnames(prov_tot) <- c("Province", "Count")

prov_tot %<>%
      mutate_at("Province",
                stringr::str_replace, "_1", "")
```

Ecoregions

```
ecoreg <- location
ecoreg_lvls <-
  unique(
    unlist(
      str_split(ecoreg$ecoreg_code, "\\_")
                                )
                          )

ecoreg_lvls<- ecoreg_lvls[! ecoreg_lvls %in% c("")]

ecoreg[,ecoreg_lvls] <- sapply(ecoreg_lvls,
                                       function(y){
                                         str_count(ecoreg$ecoreg_code, y)
                                         }
                                       )
```

Create a column with the number of ecoregions per study

```
ecoreg %<>%
   mutate(
     ecoreg_per_record = rowSums(
     across(
       where(is.numeric)
       )
     )
   )

ecoreg$ecoreg_per_record <- as.factor(ecoreg$ecoreg_per_record)
```

Calculate number of records by number of ecoregions in %

```
ecoreg_per_record_df <- ecoreg %>%
  group_by(ecoreg_per_record) %>%
  summarise(n_records = length(ecoreg_per_record)) %>%
  mutate(percentage= round(n_records / sum(n_records) * 100,1))

ecoreg_per_record_df
```

```
## # A tibble: 6 × 3
##   ecoreg_per_record n_records percentage
##   <fct>                <int>      <dbl>
## 1 1                      884       88.3
## 2 2                       74        7.4
## 3 3                       30        3
## 4 4                        9        0.9
## 5 5                        3        0.3
## 6 12                       1        0.1
```

Create a data frame with number of publication per ecoregion

```
ecoreg_tot <-
  ecoreg [, -(length(ecoreg)-1)] %>% # filter out the sum column
  summarise(across(where(is.numeric),
                   list(sum)
                   )
            )

ecoreg_tot <- reshape2::melt(ecoreg_tot, id.vars = NULL)

colnames(ecoreg_tot) <- c("Ecoregion", "Count")

ecoreg_tot %<>%
      mutate_at("Ecoregion",
                stringr::str_replace, "_1", "")
```

Add count column to Meow and set 0 to all the ecoregions where no study was conducted

```
colnames(ecoreg_tot)[colnames(ecoreg_tot) == 'Ecoregion'] <- 'ECO_CODE'
ecoreg_tot$ECO_CODE <- as.character(ecoreg_tot$ECO_CODE)
```

```
meow$ECO_CODE <- as.character(meow$ECO_CODE)



ecoreg_tot <-
  left_join(meow, ecoreg_tot, by = 'ECO_CODE') %>%
  mutate(Count = replace(Count, is.na(Count), 0))
```

Export count per ecoregion to csv for the map in QGIS

```
writexl::write_xlsx(ecoreg_tot,
           here("Extracted_data_analysis","global","Output-Transformed_data",
"eco_reg_tot.xlsx"))
```

Studies per ecoregion

```
print(paste("Percentage ecoregions with no studies:", round(nrow(subset(ecore
g_tot, ecoreg_tot$Count == 0))/nrow(ecoreg_tot)*100,1),"%"))

## [1] "Percentage ecoregions with no studies: 44 %"

print(paste("Percentage ecoregions with a single study:", round(nrow(subset(e
coreg_tot, ecoreg_tot$Count == 1))/nrow(ecoreg_tot)*100,1),"%"))

## [1] "Percentage ecoregions with a single study: 15.9 %"
```

Realms

```
realms <- ecoreg_tot %>%
    group_by(RLM_CODE, REALM) %>%
  summarise(Count=sum(Count))

## `summarise()` has regrouped the output.
## ℹ Summaries were computed grouped by RLM_CODE and REALM.
## ℹ Output is grouped by RLM_CODE.
## ℹ Use `summarise(.groups = "drop_last")` to silence this message.
## ℹ Use `summarise(.by = c(RLM_CODE, REALM))` for per-operation grouping
##   (`?dplyr::dplyr_by`) instead.

realms %<>%
  rename(Code = RLM_CODE,
         Realm = REALM)

writexl::write_xlsx(realms,
           here("Extracted_data_analysis","global","Output-Transformed_data",
"Realm_count.xlsx"))
```

Latitudinal region

Calculate the number of studies per latitude zone

```
env <- raw[,c (
  "File","latitudinalRegion")]
```

```r
unique(env$latitudinalRegion)
```

```
## [1] "tropical"           "temperate"          "polar"
## [4] "temperate+tropical" "polar+temperate"
```

```r
env_lvls <-
  unique(
    unlist(
      str_split(env$latitudinalRegion, "\\_")
                                )
                          )

env$latitudinalRegion[env$latitudinalRegion=="tropical+temperate"] <-"tempera
te+tropical"



env$latitudinalRegion <- as.factor(env$latitudinalRegion)

env[,env_lvls] <- sapply(env_lvls,
                                      function(y){
                                        str_count(env$latitudinalRegion, y)
                                        }
                                      )

env %<>% tidyr::replace_na(list(latitudinalRegion = "temperate"))

str(env)
```

```
## tibble [1,001 × 7] (S3: tbl_df/tbl/data.frame)
##  $ File              : chr [1:1001] "AbdulWahabEtAl_2018_Biodiversity_spat
ial_patterns.md" "AbesamisEtAl_2018_Benthic_habitat_fish.md" "AchilleosEtAl_2
020_Bryozoan_diversity_Cyprus.md" "AdamsEtAl_2020_Rhodolith_bed_discovered.md
" ...
##  $ latitudinalRegion : Factor w/ 5 levels "polar","polar+temperate",..: 5
5 3 3 3 3 3 5 5 3 ...
##  $ tropical          : int [1:1001] 1 1 0 0 0 0 0 1 1 0 ...
##  $ temperate         : int [1:1001] 0 0 1 1 1 1 1 0 0 1 ...
##  $ polar             : int [1:1001] 0 0 0 0 0 0 0 0 0 0 ...
##  $ temperate+tropical: int [1:1001] 0 0 0 0 0 0 0 0 0 0 ...
##  $ polar+temperate   : int [1:1001] 0 0 0 0 0 0 0 0 0 0 ...
```

```r
env_tot<- as.data.frame(summary(env$latitudinalRegion))

env_tot %<>%
  mutate(Latitude_zone = as.factor(rownames(env_tot))) %>%
  rename(Count = "summary(env$latitudinalRegion)")
```

```
env_tot

##                     Count      Latitude_zone
## polar                  29              polar
## polar+temperate         1    polar+temperate
## temperate             526          temperate
## temperate+tropical     15 temperate+tropical
## tropical              430           tropical
```

Export geographical distribution summary tables

```
writexl::write_xlsx(list(env_tot,realms, prov_tot, prov_per_record_df, ecoreg
_tot, ecoreg_per_record_df),
           here("Extracted_data_analysis","global","Output-Transformed_data",
"geographical_distribution_summary.xlsx"))
```

COUNT COLUMNS WITH COMBO

Create a subset of columns

```
count_columns <- raw %>%
  select(File,
         meso_substrate,
         habit,
         environmentalPosition,
         observationalORexperimental,
         taxonomicTargetLevel,
         indicators,
         collectedDataType,
         molecularAnalyses)
```

Prepare data set with columns that need count only and add percentage column

```
count_columns[,] <- lapply(count_columns[,], as.factor)

count_results <- list()

for (col_name in colnames(count_columns)) {
  counts <- count(count_columns, !!sym(col_name))

  count_results[[col_name]] <- counts

  colnames(count_results[[col_name]]) <- c(col_name, "Count")

  count_results[[col_name]] %<>%
    filter(
      if_all(.cols = everything(), ~ !grepl("notapp", .x))
    ) %>%
```

```
 # delete rows with "notapp"
    tidyr::drop_na()

  count_results[[col_name]] %<>%
    mutate(Percentage= round(Count / sum(Count) * 100,2))
}

rm(col_name, counts)

count_results <- count_results[names(count_results) != "File"]
```

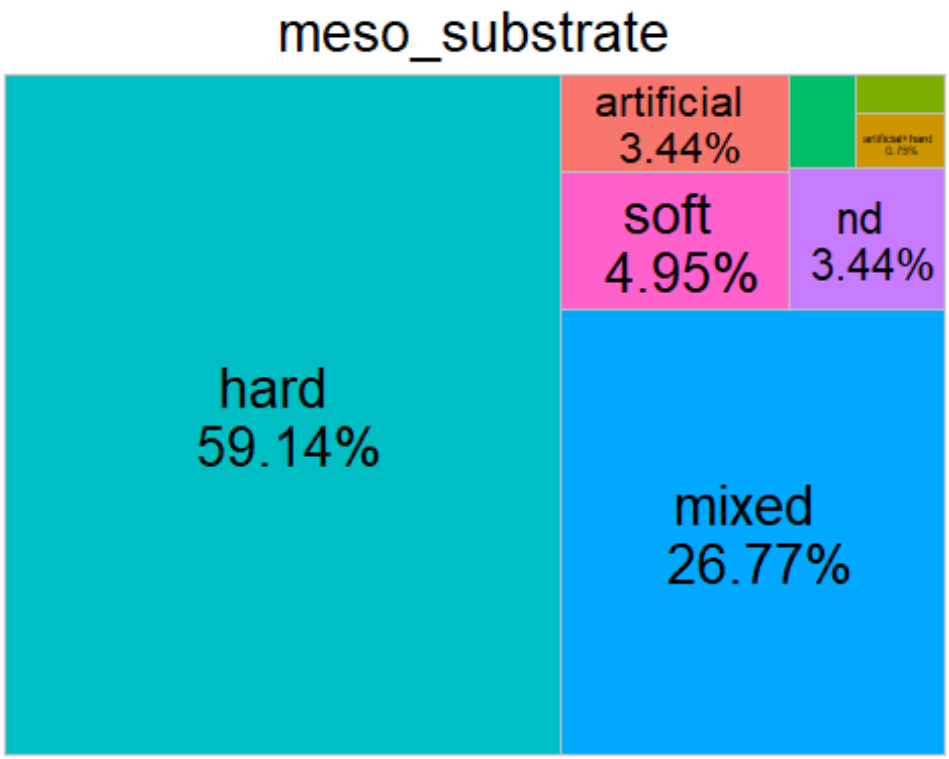
Export summary tables

```
library(openxlsx)

write.xlsx(count_results,
           file = here("Extracted_data_analysis",
                       "global",
                       "Output-Transformed_data",
                     "summary_counts_question_1-3.xlsx"))
```

Create treemaps charts

```
treemap_func = function(dat,var) {
  ggplot(dat,
       aes(area=Count,
           fill=.data[[var]],
           line = "black",
           label=paste0(.data[[var]],
                        " \n ",Percentage,"%"))
       )+
  treemapify::geom_treemap(layout = "squarified")+
  treemapify::geom_treemap_text(place = "centre",
                    size = 20)+
  labs(title = var)+
    theme(plot.title = element_text(hjust = 0.5, size= 20),
          legend.position = "none")

}

treemap_counts <- list()

for (col_name in colnames(count_columns[,-1])) {

  treemap_counts[[col_name]] <- treemap_func(dat= count_results[[col_name]],
var = col_name)

}
```

```
treemap_counts[[1]]
```



Save all tree maps plots

```
lapply(names(treemap_counts), function(x){
  ggsave(filename=paste0(x,".tiff"),
         plot=treemap_counts[[x]],
         units = "in", width=12, height=4, dpi=300, compression = 'lzw',
         path = here("Extracted_data_analysis",
                     "global",
                     "Output-Visualization",
                   "Treemap_counts"),
         create.dir = T
         )
  })

lapply(names(treemap_counts), function(x){
  ggsave(filename=paste0(x,".png"),
         plot=treemap_counts[[x]],
         units = "in", width=12, height=4, dpi=300,
         path = here("Extracted_data_analysis",
                     "global",
                     "Output-Visualization",
                   "Treemap_counts"),
         create.dir = T
```

```
        )
  })
```

COUNT METHODS (PURELY TAXONOMIC STUDIES EXCLUDED)

Remove purely taxonomic studies and subset columns (taxa++ was assigned to studies that led to the description of new species but where also ecological e.g. describing community for a given taxonomic group)

```
methods <- raw %>%
  filter(taxonomic_publication == "taxa++" |  is.na(taxonomic_publication) )
 %>%
  select(File,
         meso_design,
         meso_replicates,
         temporalReplicates,
         abioticVariablesCollection,
         abioticVariables)

count_columns_method <- methods
```

Prepare data set with columns that need count only and add percentage column

```
count_columns_method[,] <- lapply(count_columns_method[,], as.factor)

count_results_method <- list()

for (col_name in colnames(count_columns_method)) {
  counts <- count(count_columns_method, !!sym(col_name))

  count_results_method[[col_name]] <- counts

  colnames(count_results_method[[col_name]]) <- c(col_name, "Count")

  count_results_method[[col_name]] %<>%
    filter(
      if_all(.cols = everything(), ~ !grepl("notapp", .x))
    ) %>% # delete rows with "notapp"
    tidyr::drop_na()

  count_results_method[[col_name]] %<>%
    mutate(Percentage= round(Count / sum(Count) * 100,1))
}

rm(col_name, counts)
```

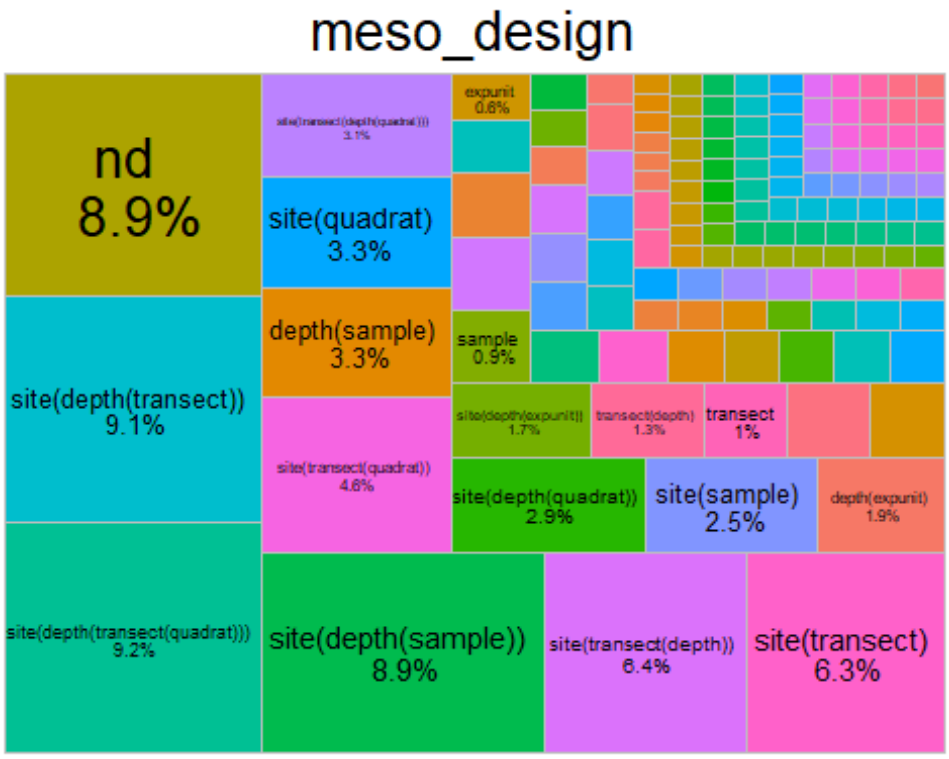
```
count_results_method <- count_results_method[names(count_results_method) != "
File"]
```

Export summary tables

```
library(openxlsx)

write.xlsx(count_results_method,
           file = here("Extracted_data_analysis",                          "glo
bal",                          "Output-Transformed_data",
                    "summary_counts_question_4.xlsx"))
```

Create treemaps charts

```
treemap_func = function(dat,var) {
  ggplot(dat,
       aes(area=Count,
           fill=.data[[var]],
           line = "black",
           label=paste0(.data[[var]],
                        " \n ",Percentage,"%"))
       )+
  treemapify::geom_treemap(layout = "squarified")+
  treemapify::geom_treemap_text(place = "centre",
                    size = 20)+
  labs(title = var)+
    theme(plot.title = element_text(hjust = 0.5, size= 20),
          legend.position = "none")

}

treemap_counts <- list()

for (col_name in colnames(count_columns_method[,-1])) {

  treemap_counts[[col_name]] <- treemap_func(dat= count_results_method[[col_n
ame]], var = col_name)

}

treemap_counts[[1]]
```



Save all tree maps plots

```
lapply(names(treemap_counts), function(x){
  ggsave(filename=paste0("treemap",x,".tiff"),
         plot=treemap_counts[[x]],
         units = "in", width=12, height=4, dpi=300, compression = 'lzw',
         path = here("Extracted_data_analysis","global",
   "Output-Visualization","Treemaps_methods"),
         create.dir = T
         )
  })

lapply(names(treemap_counts), function(x){
  ggsave(filename=paste0("treemap",x,".png"),
         plot=treemap_counts[[x]],
         units = "in", width=12, height=4, dpi=300,
         path = here("Extracted_data_analysis",                          "globa
l",                          "Output-Visualization","Treemaps_methods"),
         create.dir = T
         )
  })
```

COUNT COLUMNS SPLIT

Create a list of data frame splitting values

```r
split_columns <- raw %>%
  select(meso_substrate,
         habit,
         environmentalPosition,
         observationalORexperimental,
         taxonomicTargetLevel,
         indicators,
         collectedDataType,
         molecularAnalyses)


split_columns[,] <- lapply(split_columns[,], as.factor)
```

Prepare data set with columns

```r
split_results <- list()

for (col_name in colnames(split_columns)) {
  # Split the values of each column by "+"
  lvls <- unique(unlist(str_split(split_columns[[col_name]], "\\+")))

  # Initialize a list to store counts for each level
  split_counts <- sapply(lvls, function(y) {
    # Count how many times each unique level appears in the column
    str_count(split_columns[[col_name]], fixed(y))
    }
    )

  split_results[[col_name]] <- split_counts
}

## Warning in stri_count_fixed(string, pattern, opts_fixed = opts(pattern)):
empty
## search patterns are not supported

rm(lvls)
```

Add percentage column

```r
split_tot <- list()

for(col_name in colnames(split_columns)) {
  split_results[[col_name]] <- as.data.frame(split_results[[col_name]])

  split_results[[col_name]] %<>% # delete rows with missing values
    select(where(function(x) any(!is.na(x)))) %>%
    select(-any_of("notapp")) %>%
    tidyr::drop_na()

    split_tot[[col_name]] <- as.data.frame(split_results[[col_name]]) %>%
```

```
    summarise(across(where(is.numeric), list(sum)))

  split_tot[[col_name]]  <- reshape2::melt(split_tot[[col_name]], id.vars = N
ULL)

  colnames(split_tot[[col_name]]) <- c(col_name, "Count")

  split_tot[[col_name]] %<>%
        mutate_at(col_name,
                  stringr::str_replace, "_1", "")


  split_tot[[col_name]] %<>%
      mutate(Percentage= round(Count / nrow(split_results[[col_name]] ) * 100,
2))

}

rm(col_name)
```

Relabel indicators

```
split_tot[[6]] %<>%
  filter(indicators != "taxo") %>%
  mutate(indicators = case_when(indicators == "Abundance" ~ "Abundance (346)",
                                indicators == "Cover" ~ "Cover (312)",
                                indicators == "Diversity" ~ "Diversity  (458)",
                                indicators == "Habitat" ~ "Habitat (181)",
                                indicators == "Morphological" ~ "Morphology (2
44)",
                                indicators == "Size" ~ "Size (218)",
                                indicators == "Biomass" ~ "Biomass  (68)",
                                indicators == "Density" ~ "Density (219)",
                                indicators == "growth" ~ "Growth rate (42)",
                                indicators == "Health" ~ "Health (90)",
                                indicators == "Physiological" ~ "Physiological
 indicator (127)",
                                indicators == "ALDFGs" ~ "ALDFGs (64)",
                                indicators == "Litter" ~ "Litter (55)",
                                indicators == "Behaviour" ~ "Behaviour (40)",
                                indicators == "Trophic" ~ "Trophic (65)",
                                indicators == "impact" ~ "Impact (16)",
                                indicators == "surv" ~ "Survival (29)",
                                indicators == "geneticSequences" ~ "Genetic se
quence (160)",
                                indicators == "rugosity" ~ "Rugosity (1)",
                                indicators == "func" ~ "Functional indicators
(20)",
                                indicators == "sex" ~ "Sex (22)",
```

```
                                indicators == "Mortality" ~ "Mortality (27)",
                                indicators == "Recruitment" ~ "Recruitment (25)
",
                                indicators == "NIS" ~ "Non-indigenous species
(24)",
                                indicators == "complexity" ~ "Complexity (6)",
                                indicators == "disease" ~ "Diseases (7)",
                                indicators == "economicValue" ~ "Economic valu
e (1)",
                                indicators == "ecoser" ~ "Ecosystem services
(3)",
                                indicators == "complexity" ~ "Complexity (6)",
                                indicators == "orientation" ~ "Orientation (1)
",
                                indicators == "fecundity" ~ "Fecundity (1)",
                                TRUE ~ indicators
  ))
```

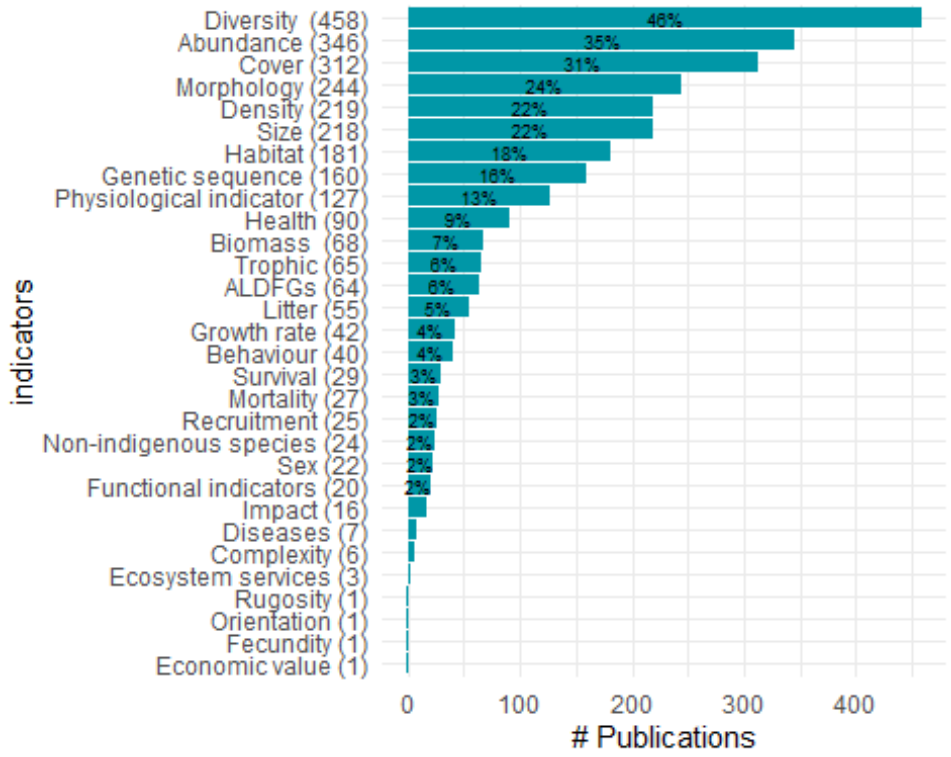
Export summary tables

```
library(openxlsx)

write.xlsx(split_tot,
           file = here("Extracted_data_analysis",                        "glo
bal",                          "Output-Transformed_data",
                    "summary_split_counts_question_1_3.xlsx"))
```

Create bar charts for each collected parameter

```
barchart_func = function(dat,var) {

 ggplot(dat,
       aes(x = reorder(.data[[var]], Count), y = Count, fill= .data[[var]]))
+
  geom_bar(stat = "identity", fill = "#0097A7") +
  geom_text(
    data = dat %>% filter(Percentage >= 2),
    aes(label = paste0(round(Percentage, 0), "%")),
    position = position_stack(vjust =0.5),
    color = "black",
    size = 2.5
  )+
  labs(y = "# Publications", x = var) +
  theme_minimal() +
  coord_flip()+  # optional: flips for horizontal bars
  theme(legend.position ="none")

}

barchart_counts <- list()
```

```
for (col_name in colnames(split_columns)) {

  barchart_counts[[col_name]] <- barchart_func(dat= split_tot[[col_name]], va
r = as.character(col_name))

}

barchart_counts[[6]]
```




```
lapply(names(barchart_counts), function(x){
  ggsave(filename=paste0("bar_",x,".tiff"),
         plot=barchart_counts[[x]],
         units = "in", width=12, height=4, dpi=300, compression = 'lzw',
         path = here("Extracted_data_analysis",                          "globa
l",                         "Output-Visualization","Barcharts_split"),
         create.dir = T
         )
  })

lapply(names(barchart_counts), function(x){
  ggsave(filename=paste0("bar_",x,".png"),
         plot=barchart_counts[[x]],
         units = "in", width=12, height=4, dpi=300,
         path = here("Extracted_data_analysis",                          "globa
l",                         "Output-Visualization","Barcharts_split"),
```

```
        create.dir = T
        )
  })
```

COUNT METHODS (PURELY TAXONOMIC STUDIES EXCLUDED)

Create data frame with subset

```
split_columns_methods <- methods

split_columns_methods[,] <- lapply(split_columns_methods[,], as.factor)
```

Prepare data set with columns

```
split_results_methods <- list()

for (col_name in colnames(split_columns_methods)) {
  # Split the values of each column by "+"
  lvls <- unique(unlist(str_split(split_columns_methods[[col_name]], "\\+")))

  # Initialize a list to store counts for each level
  split_counts_methods <- sapply(lvls, function(y) {
    # Count how many times each unique level appears in the column
    str_count(split_columns_methods[[col_name]], fixed(y))
    }
    )

  split_results_methods[[col_name]] <- split_counts_methods
}

rm(lvls)
```

Add percentage column

```
split_tot_methods <- list()

for(col_name in colnames(split_columns_methods)) {
  split_results_methods[[col_name]] <- as.data.frame(split_results_methods[[c
ol_name]])

  split_results_methods[[col_name]] %<>% # delete rows with missing values
    select(where(function(x) any(!is.na(x)))) %>%
    select(-any_of("notapp")) %>%
    tidyr::drop_na()

    split_tot_methods[[col_name]] <- as.data.frame(split_results_methods[[col
_name]]) %>%
```

```
    summarise(across(where(is.numeric), list(sum)))

  split_tot_methods[[col_name]]  <- reshape2::melt(split_tot_methods[[col_nam
e]], id.vars = NULL)

  colnames(split_tot_methods[[col_name]]) <- c(col_name, "Count")

  split_tot_methods[[col_name]] %<>%
        mutate_at(col_name,
                  stringr::str_replace, "_1", "")


  split_tot_methods[[col_name]] %<>%
      mutate(Percentage= round(Count / nrow(split_results_methods[[col_name]]
 ) * 100,2))

}

rm(col_name)
```

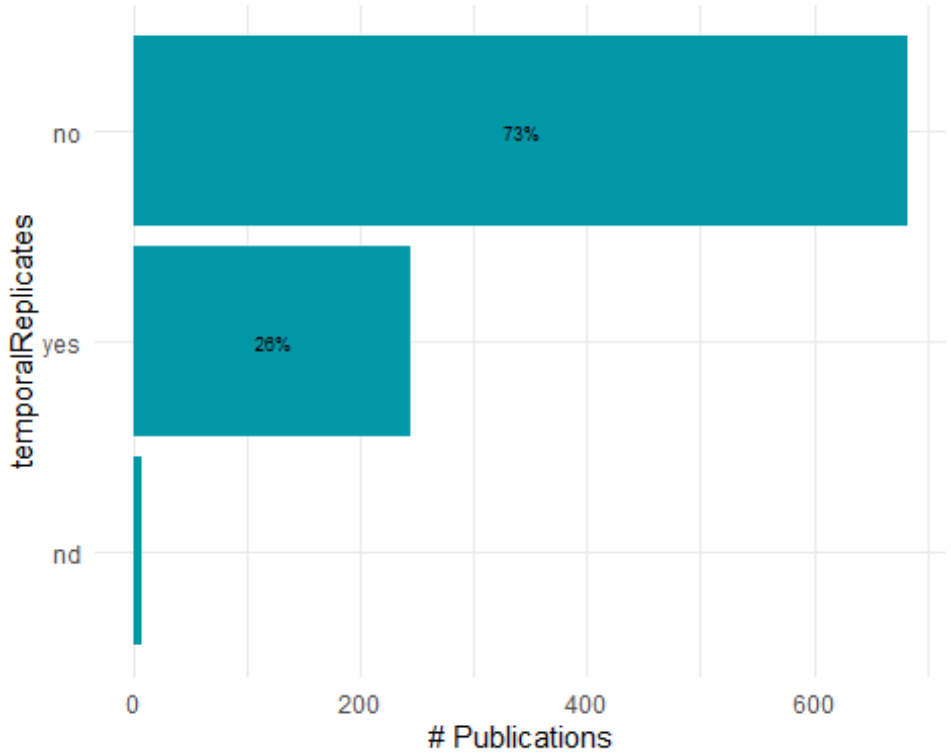
Export summary tables

```
library(openxlsx)

write.xlsx(split_results_methods,
           file = here("Extracted_data_analysis",                          "glo
bal",                          "Output-Transformed_data",
                    "summary_split_counts_question_4.xlsx"))
```

Create bar charts per each parameter collected

```
barchart_func = function(dat,var) {

 ggplot(dat,
       aes(x = reorder(.data[[var]], Count), y = Count, fill= .data[[var]]))
+
  geom_bar(stat = "identity", fill = "#0097A7") +
  geom_text(
    data = dat %>% filter(Percentage >= 2),
    aes(label = paste0(round(Percentage, 0), "%")),
    position = position_stack(vjust =0.5),
    color = "black",
    size = 2.5
  )+
  labs(y = "# Publications", x = var) +
  theme_minimal() +
  coord_flip()+  # optional: flips for horizontal bars
  theme(legend.position ="none")
```

```
}

barchart_split_methods <- list()

for (col_name in colnames(split_columns_methods)) {

  barchart_split_methods[[col_name]] <- barchart_func(dat= split_tot_methods
[[col_name]], var = as.character(col_name))

}

barchart_split_methods[[4]]
```



Export bar charts in a dedicated folder

```
lapply(names(barchart_split_methods), function(x){
  ggsave(filename=paste0("bar_",x,".tiff"),
         plot=barchart_split_methods[[x]],
         units = "in", width=12, height=4, dpi=300, compression = 'lzw',
         path = here("Extracted_data_analysis",                        "globa
l",                         "Output-Visualization","Barcharts_split_methods"),
 create.dir = T
         )
  })

lapply(names(barchart_split_methods), function(x){
```

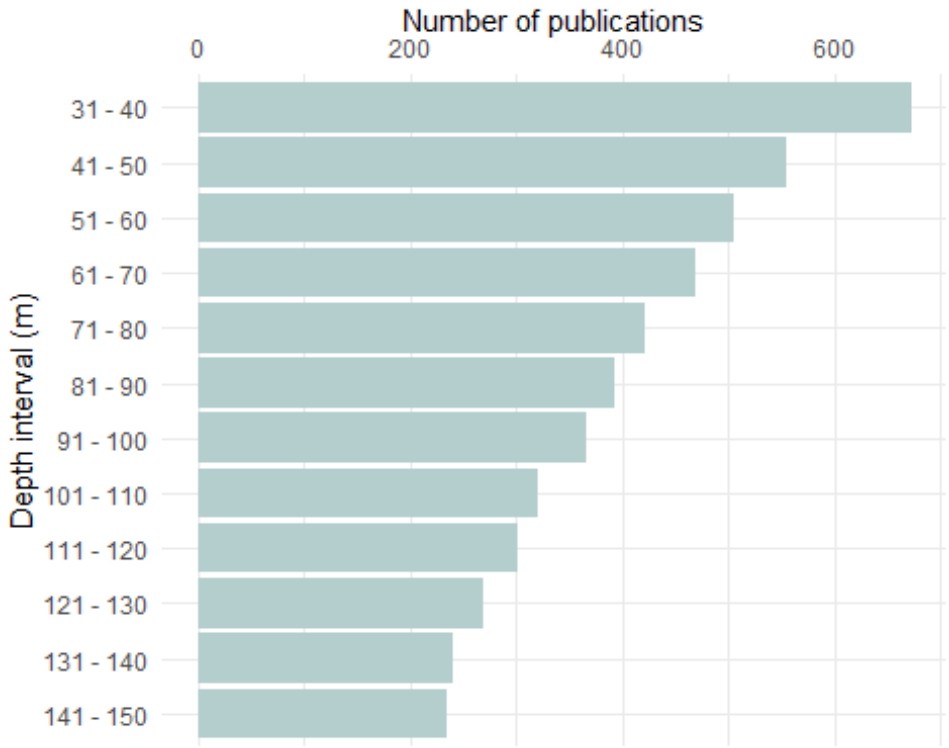
```
  ggsave(filename=paste0("bar_",x,".png"),
         plot=barchart_split_methods[[x]],
         units = "in", width=12, height=4, dpi=300,
         path = here("Extracted_data_analysis",                          "globa
l",                        "Output-Visualization","Barcharts_split_methods"),
 create.dir = T
         )
  })
```

PUBLICATIONS PER DEPTH

Publications per depth

```
raw$mininumSurveyedDepthInMeters <- as.numeric(raw$mininumSurveyedDepthInMete
rs)

## Warning: NAs introduced by coercion

raw$maximumSurveyedDepthInMeters <- as.numeric(raw$maximumSurveyedDepthInMete
rs)

## Warning: NAs introduced by coercion

# Define depth bins
bins <- seq(31, 151, by = 10)

# Create a data frame to hold bin counts
pubs_depth <- data.frame(
  bin_start = bins[-length(bins)],
  bin_end = bins[-1]-1,
  count = 0
)

# For each bin, count how many studies cover that depth range
for (i in 1:nrow(pubs_depth)) {
  bin_start <- pubs_depth$bin_start[i]
  bin_end <- pubs_depth$bin_end[i]

  pubs_depth$count[i] <- sum(raw$maximumSurveyedDepthInMeters >= bin_start &
raw$mininumSurveyedDepthInMeters < bin_end, na.rm = TRUE)
}

pubs_depth$depth_bin <- paste(pubs_depth$bin_start, "-", pubs_depth$bin_end)

pubs_depth$depth_bin <- reorder(pubs_depth$depth_bin, -pubs_depth$bin_start)


ggplot(pubs_depth, aes(x = factor(depth_bin), y = count)) +
  geom_bar(stat = "identity", fill = "lightcyan3") +
```

```
  labs(x = "Depth interval (m)", y = "Number of publications") +
  theme_minimal()+
  coord_flip()+
  scale_y_continuous(position="right")
```

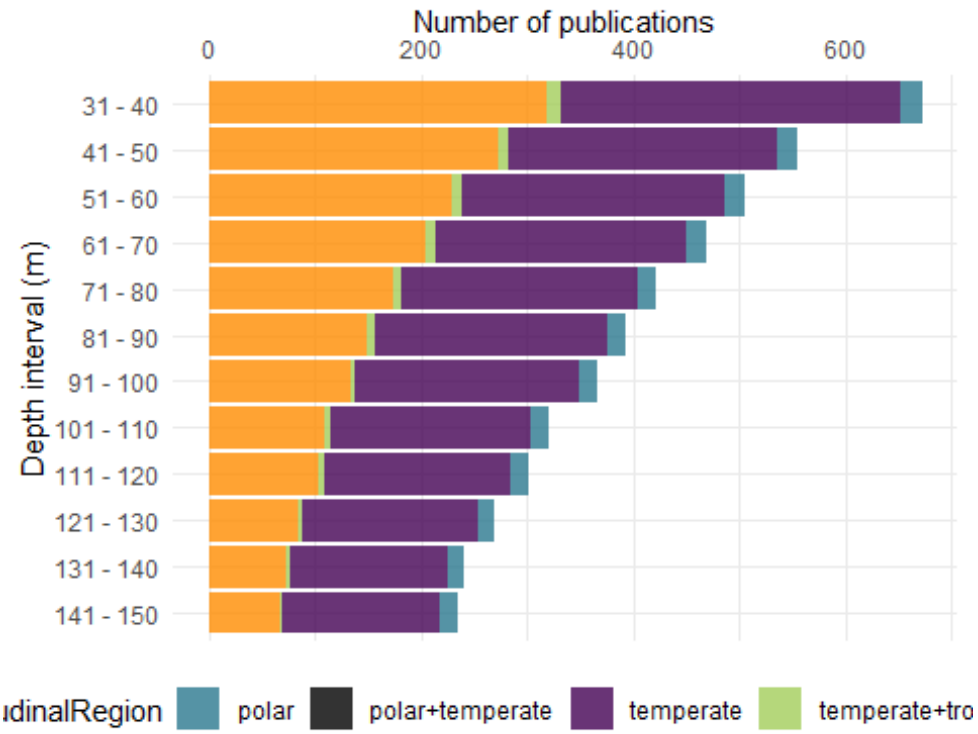


Export pubs per depth bar plot

Publication per depth by latitudinalRegion

```
# Ensure numeric values
raw$mininumSurveyedDepthInMeters <- as.numeric(raw$mininumSurveyedDepthInMete
rs)
raw$maximumSurveyedDepthInMeters <- as.numeric(raw$maximumSurveyedDepthInMete
rs)

# Define depth bins
bins <- seq(31, 150, by = 10)  # stops at 150
bin_endpoints <- bins + 9      # each bin covers 10 units (31–40, etc.)

# Create an empty results data frame
pub_depth_lat <- data.frame()

# Loop over bins and latitudinalRegions
for (i in seq_along(bins)) {
  bin_start <- bins[i]
  bin_end <- bin_endpoints[i]
```

```r
  for (lat in unique(raw$latitudinalRegion)) {
    subset_data <- raw %>% filter(latitudinalRegion == lat)

    # Count how many studies cover this bin
    count <- sum(subset_data$maximumSurveyedDepthInMeters >= bin_start &
                   subset_data$mininumSurveyedDepthInMeters < bin_end, na.rm =
TRUE)

    # Append to results
    pub_depth_lat <- rbind(pub_depth_lat, data.frame(
      depth_bin = paste(bin_start, "-", bin_end),
      latitudinalRegion = lat,
      count = count,
      bin_start = bin_start
    ))
  }
}

# Order the bins for plotting
pub_depth_lat$depth_bin <- factor(pub_depth_lat$depth_bin,
                                  levels = unique(pub_depth_lat$depth_bin[ord
er(-pub_depth_lat$bin_start)]))

# Plot
pub_depth_lat_bar <- ggplot(pub_depth_lat, aes(x = depth_bin, y = count, fill
 = latitudinalRegion)) +
  geom_bar(stat = "identity", position = "stack", alpha = 0.8) +
  labs(x = "Depth interval (m)", y = "Number of publications") +
  theme_minimal() +
  coord_flip() +
  scale_y_continuous(position = "right") +
  scale_fill_manual(values = c("#2a788e", "black", "#440154", "darkolivegreen
3", "darkorange")) +
  theme(legend.position = "bottom")

pub_depth_lat_bar
```



Export bar chart as pub_depth_env.png

PLATFORM BY DEPTH AND latitudinalRegion

Create a platform by depth data frame and keep meso_samplingPlatform

```
platform_depth_lat <- raw %>%
  select(File, mininumSurveyedDepthInMeters,maximumSurveyedDepthInMeters,
         latitudinalRegion, meso_direct_indirect,
         meso_samplingPlatform,
         max_depth_hov, min_depth_hov,
         max_depth_rov, min_depth_rov,
         max_depth_ccr, min_depth_ccr,
         max_depth_open, min_depth_open,
         max_depth_scuba, min_depth_scuba
         )
```

Split the platform column in one binary column per platform

```
platform_lat_lvls <-
  unique(
    unlist(
      str_split(platform_depth_lat$meso_samplingPlatform, "\\+")
                                )
                          )
```

```
platform_lat_lvls<- platform_lat_lvls[! platform_lat_lvls %in% c("",NA,"notap
p",",","nd")]

platform_depth_lat[,platform_lat_lvls] <- sapply(platform_lat_lvls,
                                        function(y){
                                          str_count(platform_depth_lat$meso_sam
plingPlatform, y)
                                          }
                                        )

rm(platform_lat_lvls)

platform_depth_lat[,-c(1:17)] <- lapply(platform_depth_lat[,-c(1:17)], as.num
eric)

platform_depth_lat[,c(1:17)] <- lapply(platform_depth_lat[,c(1:17)], as.chara
cter)



platform_depth_lat %<>%
  rename(open_direct = open,
         ccr_direct = ccr,
         scuba_direct = scuba)  %>%
  select(-meso_samplingPlatform)
```

Create a summary table with # studies per platform

```
platform_summary <- platform_depth_lat %>%
  summarise(across(where(is.numeric), ~ sum(.x, na.rm = TRUE)))

platform_summary <- as.data.frame(t(platform_summary))

platform_summary %<>%
  mutate(Platform = rownames(platform_summary)) %>%
  rename(Count = V1) %>%
  relocate(Count,.after = where(is.character))

platform_summary
```

```
##                   Platform Count
## bruv                  bruv    15
## tcs                    tcs    40
## rov                    rov   386
## trawl                trawl    12
## dredge              dredge    34
## scuba_direct scuba_direct   159
## fishery           fishery    43
## open_direct   open_direct   297
## hov                    hov   128
```

```
## grab                  grab    55
## ccr_direct     ccr_direct    91
## auv                    auv    29
## drop                  drop    37
## vams                  vams     2
## mixed                mixed     1
## drift                drift     2
## bdrop                bdrop     1
## station            station     2
## trap                  trap     2
```

Create a column with the number of platforms per study

```
platform_per_record <- platform_depth_lat %>%
  filter(meso_direct_indirect != "nd") %>%
   mutate(
     platform_per_record = rowSums(
     across(
       where(is.numeric)
       )
     )
   )

platform_per_record$platform_per_record <- as.factor(platform_per_record$plat
form_per_record)
```

Calculate number of records by number of platforms in %

```
platform_per_record <- platform_per_record %>%
  group_by(platform_per_record) %>%
  summarise(n_records = length(platform_per_record)) %>%
  mutate(percentage= round(n_records / sum(n_records) * 100,1))

platform_per_record
```

```
## # A tibble: 5 × 3
##   platform_per_record n_records percentage
##   <fct>                  <int>      <dbl>
## 1 1                        725       72.8
## 2 2                        211       21.2
## 3 3                         52        5.2
## 4 4                          7        0.7
## 5 5                          1        0.1
```

Keep most common visual platforms only

```
platform_depth_mod <- cbind(platform_depth_lat[,c(1:15)],
                            platform_depth_lat$ccr_direct,
                            platform_depth_lat$open_direct,
                            platform_depth_lat$scuba_direct,
                            platform_depth_lat$hov,
```

```
                                platform_depth_lat$rov)

names(platform_depth_mod) <- gsub("platform_depth_lat\\$", "", names(platform
_depth_mod))
```

Set max depth of studies with max depth > 150 to 150 so they fit into the plot

```
platform_depth_long <- within(platform_depth_mod, rm(mininumSurveyedDepthInMe
ters, maximumSurveyedDepthInMeters, max_depth_direct, meso_direct_indirect, h
ov, rov, open_direct, ccr_direct, scuba_direct))

platform_depth_long[,-c(1,2)] <- lapply(platform_depth_long[,-c(1,2)], as.num
eric)

str(platform_depth_long)

## 'data.frame':    1001 obs. of  12 variables:
##  $ File             : chr  "AbdulWahabEtAl_2018_Biodiversity_spatial_patte
rns.md" "AbesamisEtAl_2018_Benthic_habitat_fish.md" "AchilleosEtAl_2020_Bryoz
oan_diversity_Cyprus.md" "AdamsEtAl_2020_Rhodolith_bed_discovered.md" ...
##  $ latitudinalRegion: chr  "tropical" "tropical" "temperate" "temperate"
 ...
##  $ max_depth_hov    : num  NA NA NA NA NA NA NA NA NA 100 ...
##  $ min_depth_hov    : num  NA NA NA NA NA NA NA NA NA 0 ...
##  $ max_depth_rov    : num  NA NA 477 220 100 NA 707 NA NA NA ...
##  $ min_depth_rov    : num  NA NA 118 30 30 NA 100 NA NA NA ...
##  $ max_depth_ccr    : num  NA NA NA NA NA NA NA NA NA NA ...
##  $ min_depth_ccr    : num  NA NA NA NA NA NA NA NA NA NA ...
##  $ max_depth_open   : num  NA NA NA NA NA NA NA NA 40 NA ...
##  $ min_depth_open   : num  NA NA NA NA NA NA NA NA 3 NA ...
##  $ max_depth_scuba  : num  NA NA NA NA NA 40 NA NA NA NA ...
##  $ min_depth_scuba  : num  NA NA NA NA NA 30 NA 1 NA NA ...

platform_depth_long  %<>%
  mutate(max_depth_scuba = case_when(max_depth_scuba >= 150 ~ 150,
                                     TRUE ~ max_depth_scuba)
         ) %>%
  mutate(max_depth_hov = case_when(max_depth_hov >= 150 ~ 150,
                                     TRUE ~ max_depth_hov)
         ) %>%
  mutate(max_depth_rov = case_when(max_depth_rov >= 150 ~ 150,
                                     TRUE ~ max_depth_rov)
         ) %>%
  mutate(max_depth_ccr = case_when(max_depth_ccr >= 150 ~ 150,
                                     TRUE ~ max_depth_ccr)) %>%
  mutate(max_depth_open = case_when(max_depth_open >= 150 ~ 150,
                                     TRUE ~ max_depth_open))
```

Set min depth of studies with min depth <30 to 30 so they fit into the plot

```
platform_depth_long %<>%
  mutate(min_depth_scuba = case_when(min_depth_scuba <= 30 ~ 30,
                                     TRUE ~ min_depth_scuba)
         ) %>%
  mutate(min_depth_hov = case_when(min_depth_hov <= 30 ~ 30,
                                     TRUE ~ min_depth_hov)
         ) %>%
  mutate(min_depth_rov = case_when(min_depth_rov <= 30 ~ 30,
                                     TRUE ~ min_depth_rov)
         ) %>%
  mutate(min_depth_ccr = case_when(min_depth_ccr <= 30 ~ 30,
                                     TRUE ~ min_depth_ccr)) %>%
  mutate(min_depth_open = case_when(min_depth_open <= 30 ~ 30,
                                     TRUE ~ min_depth_open))
```

Convert platform_depth_long to long format

```
platform_min_depth_long <- platform_depth_long %>%
  select(-c(max_depth_hov, max_depth_rov, max_depth_open, max_depth_scuba, ma
x_depth_ccr))


platform_min_depth_long[,c(1,2)] <- lapply(platform_min_depth_long[,c(1,2)],
as.factor)

platform_min_depth_long %<>%
  reshape2::melt(platform_min_depth_long,
                 id.vars = c("File","latitudinalRegion"),
                 measure.vars = grep("^min_depth_|^max_depth_",
                                     names(platform_min_depth_long),
                                     value = TRUE),
                 variable.name = "Platform",
                 value.name = "min_depth"
                 )

platform_max_depth_long <- platform_depth_long %>%
    select(-c(min_depth_hov, min_depth_rov, min_depth_open, min_depth_scuba,
min_depth_ccr))

platform_max_depth_long %<>%
    reshape2::melt(platform_max_depth_long,
                 id.vars = c("File","latitudinalRegion"),
                 measure.vars = grep("^min_depth_|^max_depth_",
                                     names(platform_max_depth_long),
                                     value = TRUE),
                 variable.name = "Platform",
                 value.name = "max_depth"
                 )
```

```
platform_depth_lat_long <- cbind(platform_min_depth_long, platform_max_depth_
long$max_depth)

rm(platform_min_depth_long, platform_max_depth_long)

platform_depth_lat_long %<>%
  rename(max_depth =`platform_max_depth_long$max_depth`)

platform_depth_lat_long$Platform <- gsub("min_depth_", "", platform_depth_lat
_long$Platform)

platform_depth_lat_long$Platform <- gsub("max_depth_", "", platform_depth_lat
_long$Platform)
```

Set proper labelling for platform

```
platform_depth_lat_long %<>%
  mutate(Platform = case_when(Platform == "ccr" ~ "CCR",
                              Platform == "scuba" ~ "SCUBA",
                              Platform == "open" ~ "OC",
                              Platform == "rov" ~ "ROV",
                              Platform == "hov" ~ "HOV",
                              TRUE ~ Platform)
         ) %>%
  mutate(Platform = as.factor(Platform))
```

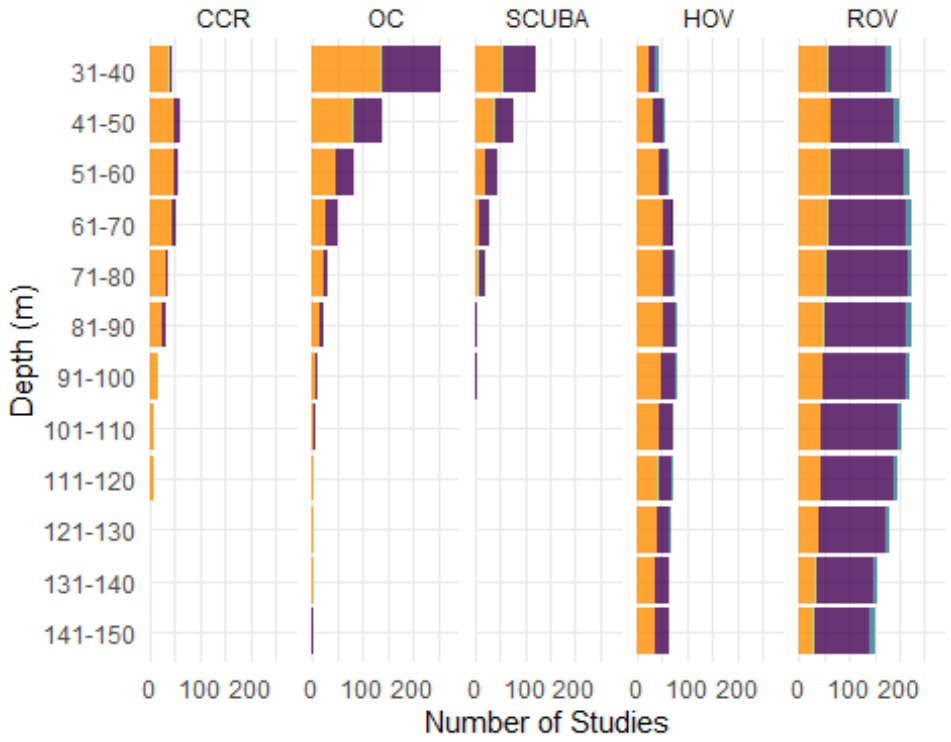
Add visual column to reorder platform

```
platform_depth_lat_long %<>%
  mutate(Visual = as.factor(case_when(Platform == "OC" |
                                      Platform == "SCUBA" |
                                      Platform == "CCR" ~ "direct",
                            TRUE ~ "indirect" )
  )
  )

platform_depth_lat_long %<>%
  mutate(Platform = forcats::fct_reorder(Platform, as.integer(Visual)))
```

Create a bar plot with depth bins by platform and latitudinalRegion

```
# Define 10-m bins between -30 and -150
bin_edges <- seq(-31, -151, by = -10)

# Create depth bin ranges and correctly ordered labels like "30-40"
depth_bins <- data.frame(
  bin_start = bin_edges[-length(bin_edges)],
  bin_end = bin_edges[-1]+1
) %>%
  mutate(
    bin_label = paste0(abs(bin_start), "-", abs(bin_end))  # e.g., "30-40"
```

```r
  )


# Expand each study to multiple rows for each bin it overlaps
expanded_data <- platform_depth_lat_long %>%
  rowwise() %>%
  mutate(
    bins = list(
      depth_bins %>%
        filter(bin_start > -max_depth & bin_end <= -min_depth)
    )
  ) %>%
  tidyr::unnest(bins)

# Ensure bin_label is an ordered factor from shallow to deep
expanded_data <- expanded_data %>%
  mutate(
    bin_label = factor(bin_label, levels = rev(depth_bins$bin_label))
  )

# Plot with count labels on the right
platform_depth_lat_plot <- ggplot(expanded_data, aes(x = bin_label, fill = la
titudinalRegion)) +
  geom_bar(position = "stack", alpha= 0.8) +
  scale_y_continuous(name = "Number of Studies",
                     expand = expansion(mult = c(0, 0.15))) +  # Make room fo
r text
  scale_x_discrete(name = "Depth (m)") +
  coord_flip() +  # Depth on y-axis (shallow at top)
  facet_wrap(~ Platform, nrow = 1) +
  theme_minimal() +
  labs(fill = "latitudinalRegion") +
  scale_fill_manual(values = c("#2a788e", "#440154", "darkolivegreen3", "dark
orange"))+
  theme(legend.position = "none")

platform_depth_lat_plot
```



Export bar plot of most common platform per depth interval by latitudinalRegion

```
ggsave("platform_depth_lat_plot.tiff",
       plot = platform_depth_lat_plot,
       units = "in", width = 8, height = 6, dpi = 300, compression = 'lzw',
       path = here ("Extracted_data_analysis",                          "global
",                        "Output-Visualization"))

ggsave("platform_depth_lat_plot.png",
       plot = platform_depth_lat_plot,
       units = "in", width = 8, height = 5,
       path = here ("Extracted_data_analysis", "global",
 "Output-Visualization"))
```

DIRECT - INDIRECT BY DECADE

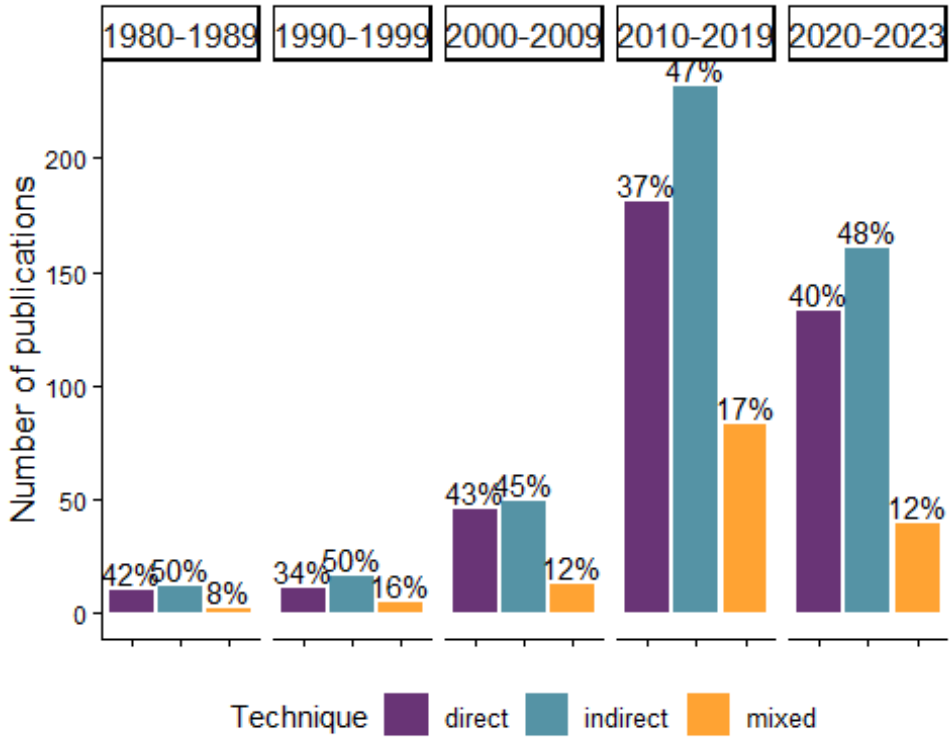
```
dir_ind_by_decade <- raw

dir_ind_by_decade %<>%
         mutate(decade = case_when(publicationYear >= 1960 & publicationYear
< 1970 ~ "1970-1979",
                                   publicationYear >= 1970 & publicationYear <
1980 ~ "1970-1979",
                                   publicationYear >= 1980 & publicationYear <
1990 ~ "1980-1989",
                                   publicationYear >= 1990 & publicationYear <
```

```
2000 ~ "1990-1999",
                                    publicationYear >= 2000 & publicationYear <
2010 ~ "2000-2009",
                                    publicationYear >= 2010 & publicationYear <
2020 ~ "2010-2019",
                                    publicationYear >= 2020 ~ "2020-2023")
              )

dir_ind_by_decade_lat <- dir_ind_by_decade

dir_ind_by_decade <- dir_ind_by_decade [, c("meso_direct_indirect","decade")]
 %>%
  group_by(decade,meso_direct_indirect) %>%
  tally()

colnames(dir_ind_by_decade) <- c("Decade", "Technique","Count")

unique(dir_ind_by_decade$Technique)

## [1] "direct"   "indirect" "mixed"    "nd"

dir_ind_by_decade %<>%
  filter(Technique != "nd" & Technique != "NA") %>% #remove nd from viz becau
se they accounted for a really small percentage
  group_by(Decade) %>%
  mutate(Percentage = round(Count / sum(Count) * 100,1))
```

Make bar plot of technique by decade

```
technique_decade_barplot <- ggplot(subset(dir_ind_by_decade,
              dir_ind_by_decade$Decade != "1970-1979"),
       aes(x=Technique, y=Count,
           fill=Technique, label = Technique)
       ) +
  geom_bar(position="dodge",
           stat="identity", alpha= 0.8) +
  geom_text(aes(label = paste(round(Percentage,0),"%", sep="")),
            parse = F,
            vjust = -0.25,
            size=3.9)+
  facet_grid(~ Decade,
             switch = "y") +
  theme_classic()+
  scale_fill_manual(values= c("#440154","#2a788e","darkorange")) +
  theme(legend.position = "bottom",
        legend.text = element_text(size = 10),
        axis.text.x = element_blank(),
        axis.text.y = element_text(size=8.7),
        axis.title.x = element_blank(),
        axis.title.y = element_text(size=13),
```

```
        strip.text = element_text(size = 13))+
  labs(y= "Number of publications")

technique_decade_barplot
```



Export bar plot of technique by decade

```
ggsave("technique_decade_barplot.tiff",
       plot = technique_decade_barplot,
       units = "in", width = 7, height = 6, dpi = 300, compression = 'lzw',
       path = here ("Extracted_data_analysis",                    "global
",                      "Output-Visualization"))

ggsave("technique_decade_barplot.png",
       plot = technique_decade_barplot,
       units = "in", width = 7, height = 6,
       path = here ("Extracted_data_analysis",                    "global
",                      "Output-Visualization"))
```

Technique by decade for temperate and tropical ecosystems

```
dir_ind_by_decade_lat <- dir_ind_by_decade_lat [, c("meso_direct_indirect","d
ecade","latitudinalRegion")] %>%
  group_by(decade,meso_direct_indirect, latitudinalRegion) %>%
  tally()

colnames(dir_ind_by_decade_lat) <- c("Decade", "Technique","latitudinalRegion
```

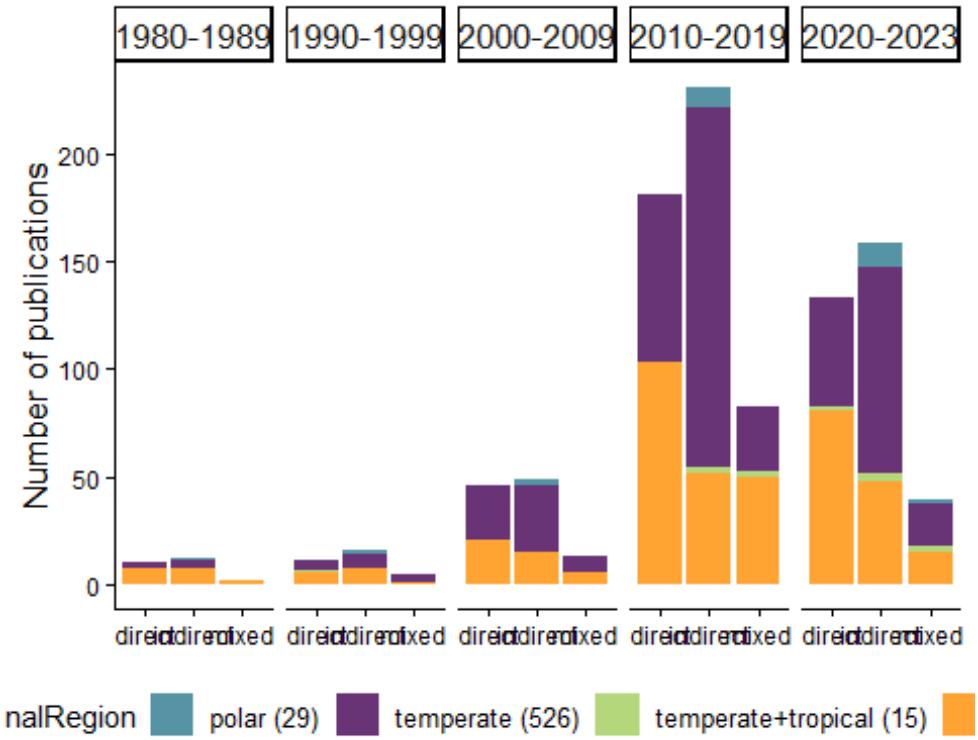
```
","Count")

unique(dir_ind_by_decade_lat$Technique)

## [1] "direct"   "indirect" "mixed"    "nd"

dir_ind_by_decade_lat %<>%
  filter(Technique != "nd" & Technique != "NA") %>% #remove nd from viz becau
se they accounted for a really small percentage
  group_by(Decade) %>%
  mutate(Percentage = round(Count / sum(Count) * 100,1))
```

Calculate publication percentage by decade and technique (not env)

```
dir_ind_by_decade_lat %<>%
  filter(Technique != "nd" & Technique != "NA") %>% #remove nd from viz becau
se they accounted for a really small percentage
  group_by(Decade) %>%
  mutate(Percentage_tech_decade = round(Count / sum(Count) * 100,1))
```

Make bar plot of technique by decade

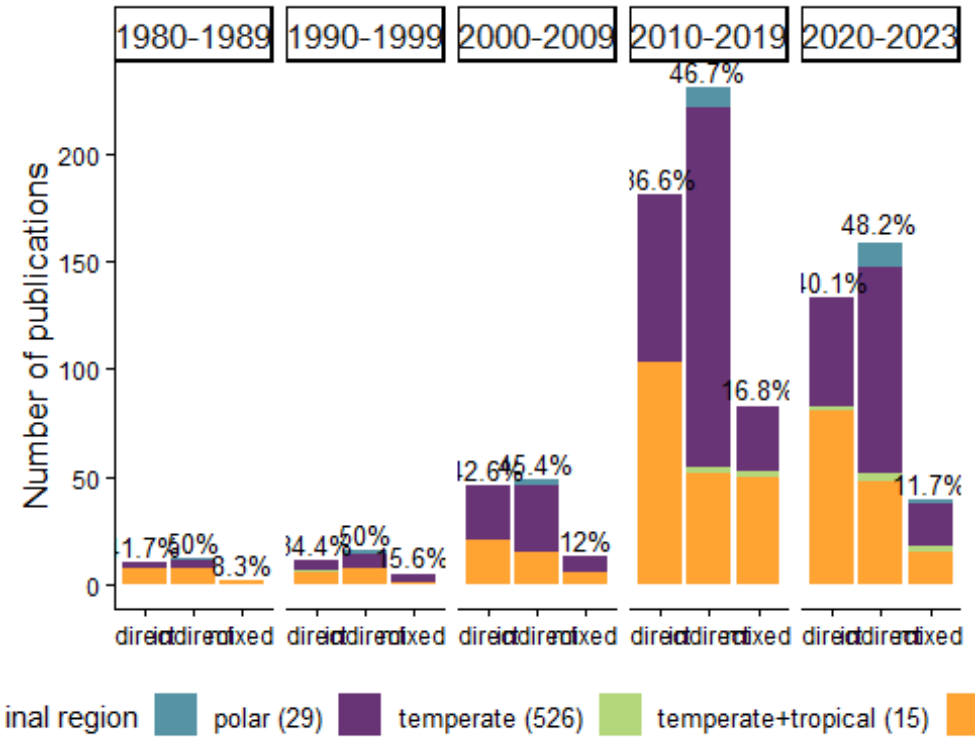
```
technique_decade__env_barplot <- ggplot(subset(dir_ind_by_decade_lat,
              dir_ind_by_decade_lat$Decade != "1970-1979" & dir_ind_by_decade
_lat$latitudinalRegion != "polar+temperate"),
       aes(x=Technique, y=Count,
           fill=latitudinalRegion, label = Technique)
       ) +
  geom_bar(position="stack",
           stat="identity", alpha= 0.8) +
  facet_grid(~ Decade,
             switch = "y") +
  theme_classic()+
  scale_fill_manual(values= c("#2a788e","#440154","darkolivegreen3","darkoran
ge"),
                    labels = c("polar (29)", "temperate (526)", "temperate+tr
opical (15)", "tropical (430)")) +
  theme(legend.position = "bottom",
        legend.text = element_text(size = 10),
        axis.text.y = element_text(size=8.7),
        axis.title.x = element_blank(),
        axis.title.y = element_text(size=13),
        strip.text = element_text(size = 13))+
  labs(y= "Number of publications")

technique_decade__env_barplot
```



Add percentage for technique and latitudinalRegion

```
# 1. Summarize label data: one row per Technique per Decade
label_data <- dir_ind_by_decade_lat %>%
  filter(Technique != "nd" & Technique != "NA") %>%
  group_by(Decade, Technique) %>%
  summarise(
    Total_Count = sum(Count),
    .groups = "drop"
  ) %>%
  group_by(Decade) %>%
  mutate(
    Percentage_label = round(Total_Count / sum(Total_Count) * 100, 1)
  )

# 2. Compute bar tops (i.e. Y position for label)
# We join with original to get full bar height, grouped by Decade and Techniq
ue
bar_tops <- dir_ind_by_decade_lat %>%
  filter(Technique != "nd" & Technique != "NA") %>%
  group_by(Decade, Technique) %>%
  summarise(
    y_position = sum(Count),
    .groups = "drop"
  )
```

```r
# 3. Merge positions with labels
label_data <- label_data %>%
  left_join(bar_tops, by = c("Decade", "Technique"))

# 4. Main bar plot data
plot_data <- dir_ind_by_decade_lat %>%
  filter(Technique != "nd" & Technique != "NA" &
         Decade != "1970-1979" & latitudinalRegion != "polar+temperate")

# 5. Filter label data for matching decades
label_data_filtered <- label_data %>%
  filter(Decade != "1970-1979")

# 6. Plot
technique_decade_env_barplot <- ggplot(plot_data,
       aes(x = Technique, y = Count, fill = latitudinalRegion)) +
  geom_bar(stat = "identity", position = "stack", alpha = 0.8) +

  # Add percentage labels above each stacked bar
  geom_text(data = label_data_filtered,
            aes(x = Technique, y = y_position,
                label = paste0(Percentage_label, "%")),
            vjust = -0.3, size = 3.5, inherit.aes = FALSE) +

  facet_grid(~ Decade, switch = "y") +
  theme_classic() +
  scale_fill_manual(
    values = c("#2a788e", "#440154", "darkolivegreen3", "darkorange"),
    labels = c("polar (29)", "temperate (526)", "temperate+tropical (15)", "t
ropical (430)")
  ) +
  theme(
    legend.position = "bottom",
    legend.text = element_text(size = 10),
    axis.text.y = element_text(size = 8.7),
    axis.title.x = element_blank(),
    axis.title.y = element_text(size = 13),
    strip.text = element_text(size = 13)
  ) +
  labs(y = "Number of publications",
       fill = "Latitudinal region")

technique_decade_env_barplot
```



Export bar plot of technique by decade

```
ggsave("technique_decade_env_barplot.tiff",
       plot = technique_decade_env_barplot,
       units = "in", width = 8, height = 6, dpi = 300, compression = 'lzw',
       path = here ("Extracted_data_analysis",                          "global
",                        "Output-Visualization"))

ggsave("technique_decade_env_barplot.png",
       plot = technique_decade_env_barplot,
       units = "in", width = 8, height = 6,
       path = here ("Extracted_data_analysis",                          "global
",                        "Output-Visualization"))
```

FIELD & INDICATORS

Create subset for field & indicators

```
field_merged <- raw[,c(59:length(raw))]

field_merged[,] <- lapply(field_merged[,], as.numeric)

colSums(field_merged)

##                  field_cons              field_commdesc
##                         121                         415
```

```
##                   field_map                  field_impass
##                         180                           255
##                 field_troph                    field_taxo
##                          69                           149
##                  field_dist                    field_aqua
##                         323                             4
##                 field_method                  field_physio
##                          63                           162
##                   field_func                 field_habtype
##                          52                             2
##                field_fishery                   field_stock
##                          54                            57
##                  field_paleo                   field_phylo
##                           1                            48
##                   field_rest                 field_archaeo
##                           9                             1
##                  field_micro             field_biomedicine
##                          10                             1
##                   field_chem       field_Molecular_ecology
##                           1                           160
##              field_Acoustics field_environmental_drivers
##                         127                           269
##              field_Behaviour           field_Geomorphology
##                         102                            57
##               field_Taxonomy
##                         128
```

Select only the most common fields

```
field_merged <- field_merged[, colSums(field_merged) > 120]

field_merged$field_Taxonomy <- NULL

colSums(field_merged)
```

```
##                   field_cons                field_commdesc
##                         121                           415
##                   field_map                  field_impass
##                         180                           255
##                 field_taxo                    field_dist
##                         149                           323
##                field_physio       field_Molecular_ecology
##                         162                           160
##              field_Acoustics field_environmental_drivers
##                         127                           269
```

```
field_merged <- field_merged[, order(colSums(field_merged), decreasing = F)]

colSums(field_merged)
```

```
##                   field_cons             field_Acoustics
##                          121                         127
##                   field_taxo     field_Molecular_ecology
##                          149                         160
##                 field_physio                   field_map
##                          162                         180
##                 field_impass field_environmental_drivers
##                          255                         269
##                   field_dist              field_commdesc
##                          323                         415
```

Rename fields with long version names

```
field_merged %<>%
  rename(`Biodiversity and \n community structure (415)` = field_commdesc,
         `Distribution and \n connectivity (323)` = field_dist,
         `Conservation and \n management (121)` = field_cons,
         `Disturbances (255)` = field_impass,
         `Mapping (180)` = field_map,
         `Physiology (162)` = field_physio,
         `Taxonomy (149)` = field_taxo,
         `Molecular ecology (160)` = field_Molecular_ecology,
         `Acoustics (127)` = field_Acoustics,
         `Environmental drivers and \n spatio-temporal patterns (269)` = fiel
d_environmental_drivers
         )
colSums(field_merged)
```

```
##                        Conservation and \n management (121)
##                                                         121
##                                              Acoustics (127)
##                                                         127
##                                               Taxonomy (149)
##                                                         149
##                                     Molecular ecology (160)
##                                                         160
##                                            Physiology (162)
##                                                         162
##                                               Mapping (180)
##                                                         180
##                                          Disturbances (255)
##                                                         255
## Environmental drivers and \n spatio-temporal patterns (269)
##                                                         269
##                         Distribution and \n connectivity (323)
##                                                         323
##                   Biodiversity and \n community structure (415)
##                                                         415
```

Prepare the indicator matrix

```
ind_matrix <- split_results[["indicators"]]
```

Select only the most common indicators

```
colSums(ind_matrix)
```

```
##        Abundance            Cover        Diversity          Habitat 
##              346              312              458              181 
##    Morphological       complexity             Size             taxo 
##              244                6              218              123 
##          Biomass           impact             surv geneticSequences 
##               68               16               29              160 
##          Density           growth           Health    Physiological 
##              219               42               90              127 
##          Trophic           ALDFGs           Litter        Behaviour 
##               65               64               55               40 
##         rugosity        Mortality              NIS      Recruitment 
##                1               27               24               25 
##             func              sex          disease    economicValue 
##               20               22                7                1 
##           ecoser      orientation        fecundity 
##                3                1                1
```

```
ind_common_matrix <- ind_matrix[, colSums(ind_matrix) > 30]

ind_common_matrix <- ind_common_matrix[, order(colSums(ind_common_matrix), de
creasing = TRUE)]

ind_common_matrix <- as.data.frame(ind_common_matrix)

ind_common_matrix <- ind_common_matrix[!purrr::map_lgl(ind_common_matrix, ~ a
ll(is.na(.)))] # remove empty column
```

Remove taxonomy studies which have a dedicated section

```
field_merged$`Taxonomy (148)` <- NULL
ind_common_matrix$taxo <- NULL

colSums(ind_common_matrix)
```

```
##        Diversity        Abundance            Cover    Morphological 
##              458              346              312              244 
##          Density             Size          Habitat geneticSequences 
##              219              218              181              160 
##    Physiological           Health          Biomass          Trophic 
##              127               90               68               65 
##           ALDFGs           Litter           growth        Behaviour 
##               64               55               42               40
```

Rename indicators with long version names

```
ind_common_matrix %<>%
  rename(`Abundance \n (346)` = Abundance,
         `Cover \n (312)` = Cover,
         `Diversity \n (458)` = Diversity,
         `Habitat \n (181)` = Habitat,
         `Morphology \n (244)` = Morphological,
         `Size \n (218) ` = Size,
         `Biomass \n (68)` = Biomass,
         `Density \n (219)` = Density,
         `Growth \n rate (42)` = growth,
         `Health \n (90)`= Health,
         `Physiological \n indicator \n (127)` = Physiological,
         `ALDFGs \n (64)` = ALDFGs,
         `Litter \n (55)` = Litter,
         `Behaviour \n (40)`= Behaviour,
         `Trophic \n (65)` = Trophic,
         `Genetic \n sequence \n (160)` = geneticSequences
         )

colSums(ind_common_matrix)
```

```
##                   Diversity \n (458)                    Abundance \n (346)
##                                  458                                   346
##                      Cover \n (312)                  Morphology \n (244)
##                                  312                                   244
##                    Density \n (219)                       Size \n (218)
##                                  219                                   218
##                    Habitat \n (181)         Genetic \n sequence \n (160)
##                                  181                                   160
## Physiological \n indicator \n (127)                      Health \n (90)
##                                  127                                    90
##                     Biomass \n (68)                     Trophic \n (65)
##                                   68                                    65
##                     ALDFGs \n (64)                      Litter \n (55)
##                                   64                                    55
##                 Growth \n rate (42)                   Behaviour \n (40)
##                                   42                                    40
```

Multiply the transpose of the field matrix with the indicator matrix

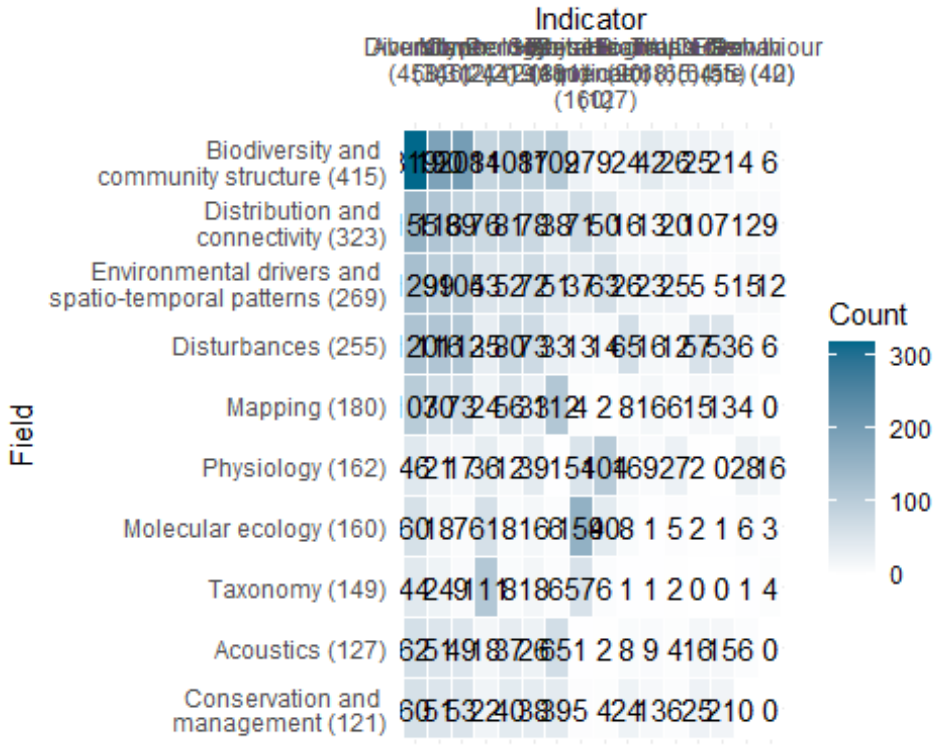
```
co_occurrence_matrix <- t(as.matrix(field_merged)) %*% as.matrix(ind_common_m
atrix)
```

Reshape the co-occurrence matrix for plotting}

```
co_occurrence_df <- reshape2::melt(co_occurrence_matrix)

colnames(co_occurrence_df) <- c("Field", "Indicator", "Count")
```

Plot the heatmap

```
field_ind_heat <- ggplot(co_occurrence_df, aes(x = Indicator, y = Field, fill
 = Count)) +
  geom_tile(color = "white") +
  scale_fill_gradient(low = "white", high = "deepskyblue4") +
  geom_text(aes(label = Count), color = "black") +
  theme_minimal()+
  scale_x_discrete(position="top")

field_ind_heat
```



Export plot

```
ggsave("field_ind_heat.png",
       units = "in", width = 13.5, height = 5.5,
       path = here ("Extracted_data_analysis", "global",
   "Output-Visualization"))
```

Calculate # of ecological indicators per record

Create a column with the sum of ecological indicators values to see how many different sampling units were adopted by each study

```
ind_summary <- as.data.frame(split_results[["indicators"]])

ind_summary <- ind_summary[!purrr::map_lgl(ind_summary, ~ all(is.na(.)))] # r
emove empty column
```

```
ind_summary %<>%  rowwise() %>%
   mutate(ind_per_record =  sum(c_across(where(is.numeric))))
```

Calculate number of records by number of sampling units in %

```
ind_summary$ind_per_record <- as.factor(ind_summary$ind_per_record)

ind_per_record <- ind_summary %>%
  group_by(ind_per_record) %>%
  summarise(n_records = length(ind_per_record)) %>%
  mutate(percentage= round(n_records / sum(n_records) * 100,1))

ind_per_record
```

```
## # A tibble: 10 × 3
##    ind_per_record n_records percentage
##    <fct>               <int>      <dbl>
##  1 0                       3        0.3
##  2 1                     125       12.5
##  3 2                     289       28.9
##  4 3                     285       28.5
##  5 4                     156       15.6
##  6 5                      85        8.5
##  7 6                      27        2.7
##  8 7                      24        2.4
##  9 8                       5        0.5
## 10 9                       2        0.2
```

SAMPLING UNIT

Prepare a data frame for sampling unit

```
sampling_df <- raw %>%
  filter(is.na(taxonomic_publication)) %>% # remove purely taxonomic studies
  select(File,recordTitle,DOI,publicationYear,mininumSurveyedDepthInMeters,ma
ximumSurveyedDepthInMeters,samplingUnit)

sampling_df %>% tidyr::drop_na(samplingUnit)
sampling_df[!is.na(sampling_df$samplingUnit),]

sampling_df %<>%
  filter(samplingUnit != "notapp")

sampling_df$samplingUnit <- as.character(sampling_df$samplingUnit)
```

Prepare columns per sampling units

```
sampling_lvls <- unique(unlist(str_split(sampling_df$samplingUnit,
                                         "\\+")
```

```
                                )
                        )

rownames(is.na(sampling_df$samplingUnit))

## NULL

sampling_df[,sampling_lvls] <- sapply(sampling_lvls,
                                      function(y){
                                        str_count(sampling_df$samplingUnit, y)
                                        }
                                      )

rm(sampling_lvls)

colnames(sampling_df)

##  [1] "File"                          "recordTitle"
##  [3] "DOI"                           "publicationYear"
##  [5] "mininumSurveyedDepthInMeters" "maximumSurveyedDepthInMeters"
##  [7] "samplingUnit"                  "quadrat"
##  [9] "sample"                        "transect"
## [11] "expunit"                       "nd"
## [13] "permquad"                      "pit"
## [15] "permtrans"                     "lit"
## [17] "stationary"                    "permlit"
## [19] "roaming_transect"              "random_swim"
## [21] "radial_transect"               "arms"
## [23] "plotless"                      "circular_transect"
```

Calculate # of sampling unit per record

Create a column with the sum of sampling units values to see how many different sampling units were adopted by each study

```
sampling_df %<>%
   mutate(type_per_record = rowSums(across(quadrat:permlit)
                        )
   )
```

Calculate number of records by number of sampling units in %

```
sampling_df$type_per_record <- as.factor(sampling_df$type_per_record)

sampling_unit_per_record <- sampling_df %>%
  group_by(type_per_record) %>%
  summarise(n_records = length(type_per_record)) %>%
  mutate(percentage= round(n_records / sum(n_records) * 100,0))

sampling_unit_per_record
```

```
## # A tibble: 5 × 3
##   type_per_record n_records percentage
##   <fct>                <int>      <dbl>
## 1 0                        1          0
## 2 1                      741         81
## 3 2                      150         16
## 4 3                       19          2
## 5 4                        1          0
```

Create a data frame with number of publication per sampling unit type

```
sampling_unit_tot <- sampling_df  %>%
  summarise(across(c(8:(length(sampling_df)-1)),
                   list(sum)))

sampling_unit_tot <- reshape2::melt(sampling_unit_tot)

## No id variables; using all as measure variables

colnames(sampling_unit_tot) <- c("Sampling_unit", "Count")

sampling_unit_tot %<>%
        mutate_at("Sampling_unit",
                  stringr::str_replace, "_1", "")
```

Simplified sampling units

Create a column with a simplified sampling units for transect and quadrat

```
  sampling_unit_tot %<>%
       mutate(Sampling_unit_macro = case_when(Sampling_unit == 'quadrat' |
                                    Sampling_unit == "permquad" ~ "Quadrat",
                                    Sampling_unit == "transect"|
                                    Sampling_unit == "lit" |
                                    Sampling_unit == "pit" |
                                    Sampling_unit == "permlit" |
                                    Sampling_unit == "permtrans" |
                                    Sampling_unit == "circular_transect" |
                                    Sampling_unit == "radial_transect"~ "Transe
ct",
                                    Sampling_unit == "nd" ~ "ND",
                                    Sampling_unit == "stationary" ~ "Stationary
",
                                    Sampling_unit == "expunit" ~ "Experimental
unit",
                                    Sampling_unit == "sample" ~ "Sample",
                                    .default = "Other")
              )

# Change columns order
```

```r
sampling_unit_tot <- cbind(sampling_unit_tot$Sampling_unit_macro,
                           sampling_unit_tot$Sampling_unit,
                           sampling_unit_tot$Count)

colnames(sampling_unit_tot) <- c("Sampling_unit_macro","Sampling_unit","Count
")

sampling_unit_tot <- as.data.frame(sampling_unit_tot)

sampling_unit_tot[,1:2] <- lapply(sampling_unit_tot[,1:2], as.factor)
sampling_unit_tot$Count <- as.numeric(sampling_unit_tot$Count)

sampling_unit_simplified <- aggregate(Count ~ Sampling_unit_macro, data = sam
pling_unit_tot, FUN = sum)

sampling_unit_simplified %<>%
  group_by(Sampling_unit_macro) %>%
  mutate(Percentage_macro= round(Count / sum(sampling_unit_simplified$Count)
* 100,1))

sampling_unit_simplified
```

```
## # A tibble: 7 × 3
## # Groups:   Sampling_unit_macro [7]
##   Sampling_unit_macro Count Percentage_macro
##   <fct>               <dbl>            <dbl>
## 1 Experimental unit      66              5.9
## 2 ND                     45              4
## 3 Other                  13              1.2
## 4 Quadrat               316             28.3
## 5 Sample                298             26.7
## 6 Stationary             18              1.6
## 7 Transect              362             32.4
```

Build a treemap with simplified sampling unit

```r
library(treemapify)

RdYlBu <- RColorBrewer::brewer.pal(7, "RdYlBu")
RdYlBu
```

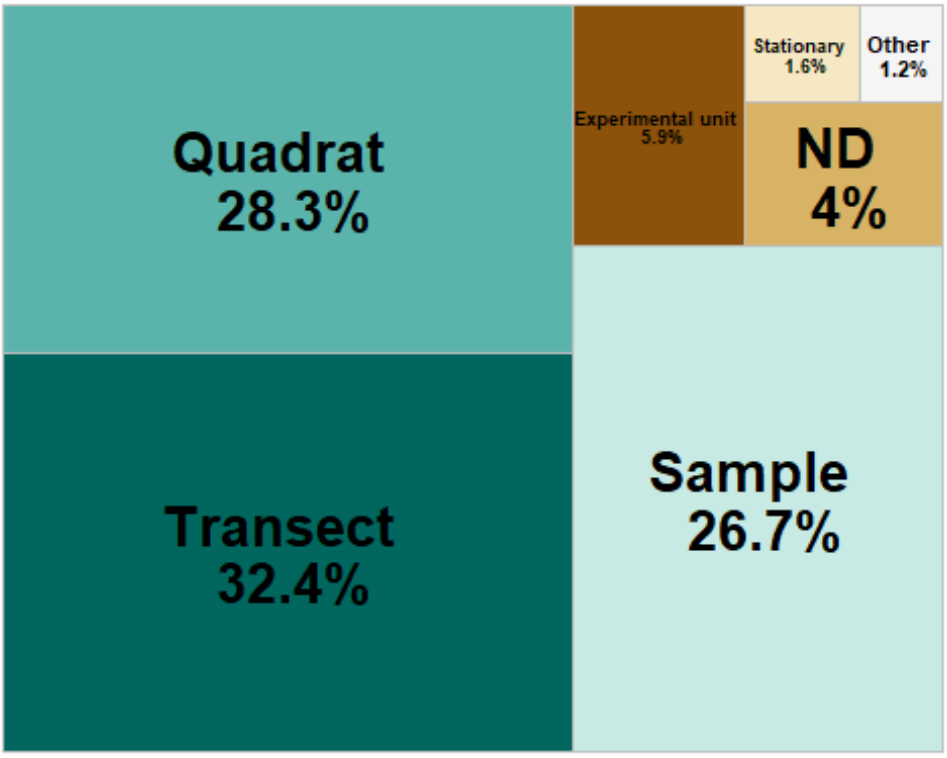
```
## [1] "#D73027" "#FC8D59" "#FEE090" "#FFFFBF" "#E0F3F8" "#91BFDB" "#4575B4"
```

```r
sampling_simplified_treemap <-
ggplot(sampling_unit_simplified,
       aes(area=Count,
           fill=Sampling_unit_macro,
           line = "black",
           label=paste0(Sampling_unit_macro, " \n ",Percentage_macro,"%"))
       )+
```

```
  treemapify::geom_treemap(layout = "squarified")+
  geom_treemap_text(place = "centre",
                    size = 20)+
    theme(plot.title = element_text(hjust = 0.5, size= 20),
          legend.position = "none")+
  scale_fill_manual(values= c("#FC8D59", "#FEE090","#FFFFBF", "#E0F3F8", "#D7
3027", "#91BFDB", "#4575B4"))
  scale_fill_brewer(palette = "RdYlBu")
```

```
## <ggproto object: Class ScaleDiscrete, Scale, gg>
##     aesthetics: fill
##     axis_order: function
##     break_info: function
##     break_positions: function
##     breaks: waiver
##     call: call
##     clone: function
##     dimension: function
##     drop: TRUE
##     expand: waiver
##     fallback_palette: function
##     get_breaks: function
##     get_breaks_minor: function
##     get_labels: function
##     get_limits: function
##     get_transformation: function
##     guide: legend
##     is_discrete: function
##     is_empty: function
##     labels: waiver
##     limits: NULL
##     make_sec_title: function
##     make_title: function
##     map: function
##     map_df: function
##     minor_breaks: waiver
##     n.breaks.cache: NULL
##     na.translate: TRUE
##     na.value: NA
##     name: waiver
##     palette: function
##     palette.cache: NULL
##     position: left
##     range: environment
##     rescale: function
##     reset: function
##     train: function
##     train_df: function
##     transform: function
```

```
##     transform_df: function
##     super:  <ggproto object: Class ScaleDiscrete, Scale, gg>

sampling_unit_simplified %<>%
  mutate(case_when(Sampling_unit_macro == "Experimental unit" ~
          "Experimental \n unit", T ~ Sampling_unit_macro))

sampling_simplified_treemap <-
ggplot(sampling_unit_simplified,
       aes(area=Count,
           fill= reorder(Sampling_unit_macro, -Count),
           line = "black",
           label=paste0(Sampling_unit_macro, " \n ",Percentage_macro,"%"))
       )+
  treemapify::geom_treemap(layout = "squarified")+
  geom_treemap_text(place = "centre", fontface = "bold",
                    size = 20)+
    theme(plot.title = element_text(hjust = 0.5, size= 20),
          legend.position = "none")+
  scale_fill_manual(values=c("#01665E","#5AB4AC","#C7EAE5", "#8C510A","#D8B36
5","#F6E8C3","#F5F5F5")) # selected from "BrBG" palette of RBrewer

sampling_simplified_treemap
```



Export sampling_simplified_treemap

```
ggsave("sampling_simplified_treemap.png",
       plot = sampling_simplified_treemap,
       units = "in", width = 7, height = 6,
       path = here ("Extracted_data_analysis",
                    "global",
                    "Output-Visualization"))
```

Export sampling_unit_tot table for supplementary materials

```
library(openxlsx)

openxlsx::write.xlsx(sampling_unit_tot,
           file = here("Extracted_data_analysis",                          "glo
bal",                          "Output-Transformed_data",
                    "sampling_unit_tot.xlsx"))
```

Quadrat extracted from transect

```
print(paste("Number of publications where quadrat is the sampling unit or one
 of the sampling unit: ",
            nrow(quad_design <- subset(raw,grepl("quad", samplingUnit)
                                       )
                               )
            )
)
```

```
## [1] "Number of publications where quadrat is the sampling unit or one of t
he sampling unit:  315"
```

```
print(paste("Number of publications where quadrats were collected along trans
ects: ", nrow(subset(quad_design,
            grepl("transect\\(quadrat", meso_design) |
              grepl("transect\\(depth\\(quadrat", meso_design)
            )
     )
)
)
```

```
## [1] "Number of publications where quadrats were collected along transects:
  209"
```

DATA TYPE

```
data_type <- count_results[["collectedDataType"]]

data_type$Percentage <- round(data_type$Percentage, 1)

data_type
```

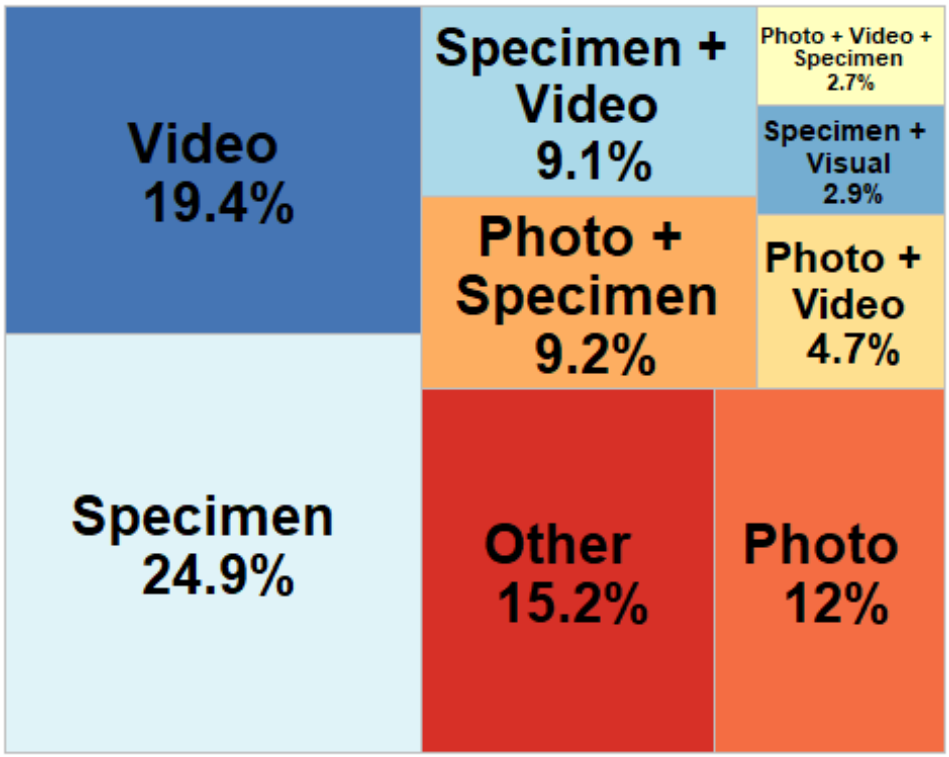
```
## # A tibble: 37 × 3
##    collectedDataType              Count Percentage
##    <fct>                          <int>      <dbl>
##  1 nd                                 1        0.1
##  2 photo                            120       12
##  3 photo+sediment+video               2        0.2
##  4 photo+specimen                    92        9.2
##  5 photo+specimen+sediment            1        0.1
##  6 photo+specimen+sediment+video      3        0.3
##  7 photo+specimen+sediment+visual     2        0.2
##  8 photo+specimen+video              27        2.7
##  9 photo+specimen+video+visual        6        0.6
## 10 photo+specimen+video+water         2        0.2
## # ℹ 27 more rows
```

Lump uncommon combination into “Other”

```
data_type_simplified <- data_type %>%
  rename(`Data type`= collectedDataType) %>%
  mutate(`Data type` = as.character(`Data type`)) %>%
  filter(Percentage > 2) %>%
  bind_rows(
    data_type %>%
      filter(Percentage <= 2) %>%
      summarise(
        `Data type` = "other",
        Count = sum(.data$Count),
        Percentage = sum(.data$Percentage)
      )
  )


data_type_simplified %<>%
       mutate(`Data type` =
                case_when(`Data type` == 'photo'  ~ "Photo",
                          `Data type` == "video"~ "Video",
                          `Data type` == "other" ~ "Other",
                          `Data type` == "specimen" ~ "Specimen",
                          `Data type` == "photo+specimen" ~ "Photo + \n Speci
men",
                          `Data type` == "photo+specimen+video" ~ "Photo + Vi
deo + \n Specimen",
                          `Data type` == "photo+video" ~ "Photo + \n Video",
                          `Data type` == "photo+visual" ~ "Photo + \n Visual",
                          `Data type` == "specimen+visual" ~ "Specimen + \n V
isual",
                          `Data type` == "specimen+video" ~ "Specimen + \n Vi
deo",
                                  .default = "Other")
              )
```

Build a treemap with data type

```
library(treemapify)
library(RColorBrewer)

data_type_treemap <-
ggplot(data_type_simplified,
       aes(area=Count,
           fill=`Data type`,
           line = "black",
           label=paste0(`Data type`, " \n ",Percentage,"%"))
       )+
  treemapify::geom_treemap(layout = "squarified")+
  geom_treemap_text(place = "centre", fontface = "bold",
                    size = 20)+
    theme(plot.title = element_text(hjust = 0.5, size= 20),
          legend.position = "none")+
    scale_fill_brewer(palette = "RdYlBu")

data_type_treemap
```



Export sampling_simplified_treemap

```
ggsave("data_type_treemap.png",
       plot = data_type_treemap,
       units = "in", width = 7, height = 6,
```

```
       path = here ("Extracted_data_analysis",                          "global
",                          "Output-Visualization"))
```

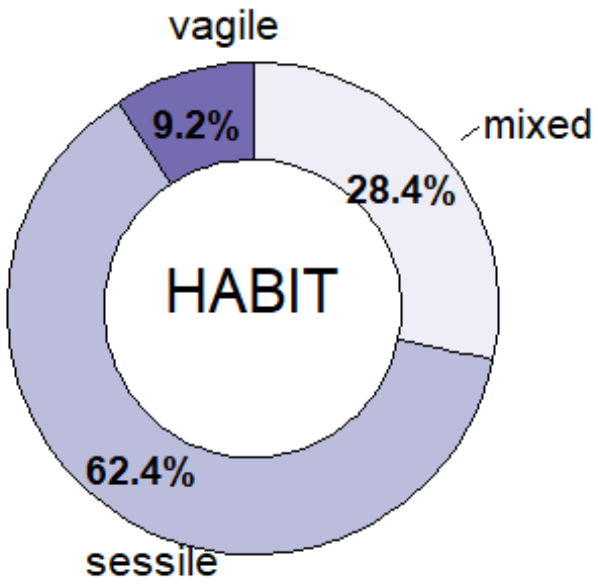
HABIT

```
habit <- as.data.frame(count_results["habit"])

# Rename columns
habit %<>%
  rename(Habit = habit.habit,
         Count = habit.Count,
         Percentage = habit.Percentage)
```

Make the habit donut plot

```
habit %<>%
  mutate(ymax = cumsum(Count),
    ymin = c(0, head(ymax, n = -1)),
    mid = (ymin + ymax) / 2,
    angle = 90 - (mid / sum(Count)) * 360
  )

# Split and assign labels
habit_inside <- habit %>%
  mutate(label = paste0(round(Percentage,1), "%"))

habit_outside <- habit %>%
  mutate(label = Habit)

# Plot
habit_donut <- ggplot() +
  # Donut segments
  geom_rect(data = habit,
            aes(ymin = ymin, ymax = ymax, xmin = 2.5, xmax = 4, fill = Habit),
            color = "black") +

  # Inside labels
  geom_text(data = habit_inside,
            aes(x = 3.2, y = mid, label = label),
            size = 5, color = "black",
            fontface = "bold") +

  # Outside labels with Habit name
  ggrepel::geom_text_repel(data = habit_outside,
                  aes(x = 4.3, y = mid, label = label),
                  size = 5.5,
                  nudge_x = 0.5,
                  direction = "y",
                  segment.color = "black",
```

```
                  hjust = 0,
                  box.padding = 0.3,
                  max.overlaps = Inf) +
  coord_polar(theta = "y") +
  xlim(0.2, 5) +
  scale_fill_brewer(palette = "Purples") +
  theme_void() +
  theme(legend.position = "none")+
  annotate(geom = 'text', x = 0.5, y = 0,
           label = c("HABIT"), size = 8)

habit_donut
```



Export habit_donut

```
ggsave("habit_donut.png",
       plot = habit_donut,
       units = "in", width = 7, height = 6,
       path = here ("Extracted_data_analysis",                    "global
",                      "Output-Visualization"))
```

SUBSTRATE

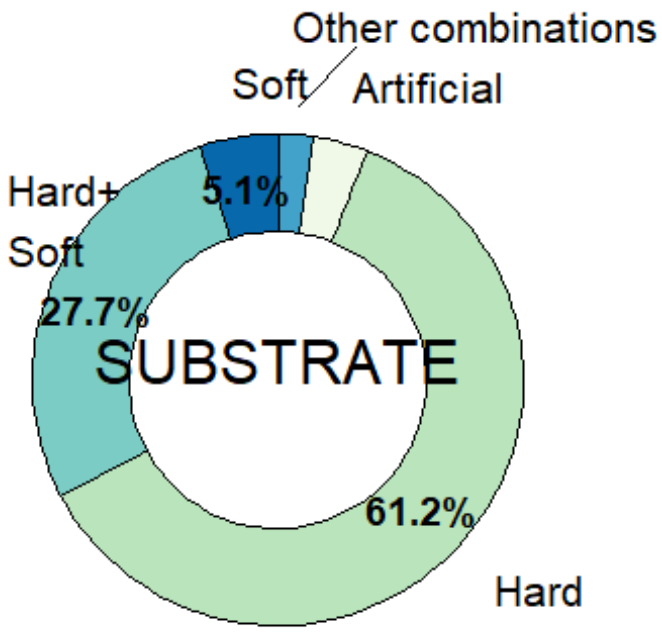
```
substrate <- as.data.frame(count_results["meso_substrate"])

# Rename columns
```

```
substrate %<>%
  rename(Substrate = meso_substrate.meso_substrate,
         Count = meso_substrate.Count,
         Percentage = meso_substrate.Percentage)

# Remove rows with nd
substrate <- substrate[!substrate$Substrate %in% "nd",]

# Merge substrate combination with less than 3% and recalculate Count and Per
centages
substrate <- substrate %>%
  mutate(Substrate = if_else(Percentage < 3, "Other combinations", Substrate))
 %>%
  group_by(Substrate) %>%
  select(-Percentage) %>%
  summarise(
    Count = sum(Count)) %>%
  mutate(Percentage = round(Count / sum(Count) * 100,1)
  )

substrate %<>%
  mutate(
    Substrate = case_when(Substrate == "hard" ~ "Hard",
                          Substrate == "soft"~ "Soft",
                          Substrate == "mixed" ~ paste0("Hard+","\n","Soft"),
                          Substrate == "artificial" ~ "Artificial",
                          Substrate == "Other combinations" ~ "Other combinat
ions"))
```

Make the substrate donut plot

```
substrate %<>%
  mutate(
    ymax = cumsum(Count),
    ymin = c(0, head(ymax, n = -1)),
    mid = (ymin + ymax) / 2,
    angle = 90 - (mid / sum(Count)) * 360
  )


# Split and assign labels
substrate_inside <- substrate %>%
    filter(Percentage >= 5) %>%
  mutate(label = paste0(Percentage, "%"))

substrate_outside <- substrate %>%
  mutate(label = Substrate)

# Plot
```

```r
substrate_donut <- ggplot() +
  # Donut segments
  geom_rect(data = substrate,
            aes(ymin = ymin, ymax = ymax, xmin = 2.5, xmax = 4, fill = Substr
ate),
            color = "black") +

  # Inside labels
  geom_text(data = substrate_inside,
            aes(x = 3.2, y = mid, label = label),
            size = 5, color = "black",
            fontface = "bold") +

  # Outside labels with substrate name
  ggrepel::geom_text_repel(data = substrate_outside,
                  aes(x = 4.4, y = mid, label = label),
                  size = 5.5,
                  nudge_x = 0.3,
                  direction = "y",
                  segment.color = "black",
                  hjust = 0,
                  box.padding = 0.3,
                  max.overlaps = Inf) +
  coord_polar(theta = "y") +
  xlim(0.2, 5) +
  scale_fill_brewer(palette = "GnBu") +
  theme_void() +
  theme(legend.position = "none")+
  annotate(geom = 'text', x = 0.5, y = 0,
           label = c("SUBSTRATE"), size = 8)

substrate_donut
```



Export the substrate donut plot

```
ggsave("substrate_donut.png",
       plot = substrate_donut,
       units = "in", width = 7, height = 6,
       path = here ("Extracted_data_analysis",                    "global
",                        "Output-Visualization"))
```

BENTHIC POSITION

Create a data frame with benthic position

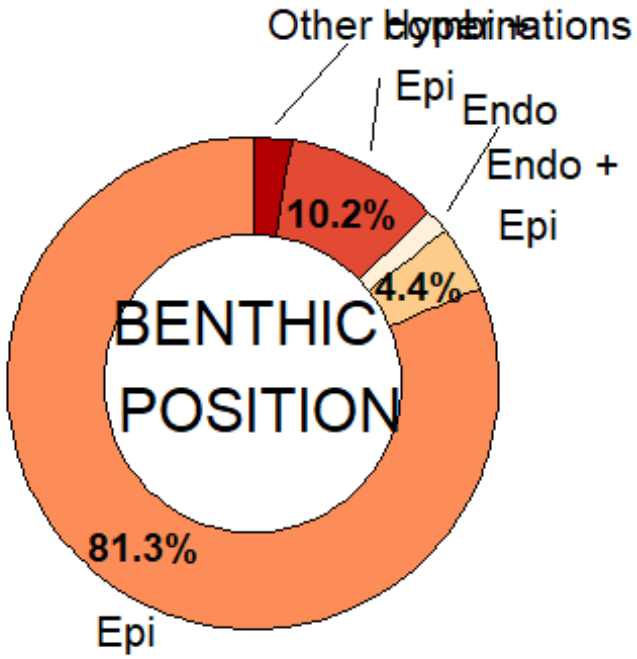
```
position <- as.data.frame(count_results["environmentalPosition"])

# Rename columns
position %<>%
  rename(Position = environmentalPosition.environmentalPosition,
         Count = environmentalPosition.Count,
         Percentage = environmentalPosition.Percentage)

# Merge position combination with less than 3% and recalculate Count and Perc
entages
position <- position %>%
  mutate(Position = if_else(Percentage < 1, "Other combinations", Position))
 %>%
  group_by(Position) %>%
```

```
  select(-Percentage) %>%
  summarise(
    Count = sum(Count)) %>%
  mutate(Percentage = round(Count / sum(Count) * 100,1)
  )
```

Make the benthic position donut plot

```
position %<>%
  mutate(
    Position = case_when(Position == "epi" ~ "Epi",
                         Position == "endo"~ "Endo",
                         Position == "dem+epi" ~ "Hyper + \n Epi",
                         Position == "endo+epi" ~ "Endo + \n Epi",
                         Position == "Other combinations" ~ "Other combinati
ons"),
    ymax = cumsum(Count),
    ymin = c(0, head(ymax, n = -1)),
    mid = (ymin + ymax) / 2,
    angle = 90 - (mid / sum(Count)) * 360
  )


# Split and assign labels
position_inside <- position %>%
  filter(Percentage > 3) %>%
  mutate(label = paste0(Percentage, "%"))

position_outside <- position %>%
  mutate(label = Position)

# Plot
benthic_position_donut <- ggplot() +
  # Donut segments
  geom_rect(data = position,
            aes(ymin = ymin, ymax = ymax, xmin = 2.5, xmax = 4, fill = Positi
on),
            color = "black") +

  # Inside labels
  geom_text(data = position_inside,
            aes(x = 3.2, y = mid, label = label),
            size = 5, color = "black",
            fontface = "bold") +

  # Outside labels with Position name
  ggrepel::geom_text_repel(data = position_outside,
                  aes(x = 4.2, y = mid, label = label),
                  size = 5.5,
```

```
                  nudge_x = 0.3,
                  direction = "y",
                  segment.color = "black",
                  hjust = 0,
                  box.padding = 0.3,
                  max.overlaps = Inf) +
  coord_polar(theta = "y") +
  xlim(0.2, 5) +
  scale_fill_brewer(palette = "OrRd") +
  theme_void() +
  theme(legend.position = "none")+
  annotate(geom = 'text', x = 0.5, y = 0,
           label = c("BENTHIC \n POSITION"), size = 8)

benthic_position_donut
```



Export the benthic_position donut plot

```
ggsave("benthic_position_donut.png",
       plot = benthic_position_donut,
       units = "in", width = 7, height = 6,
       path = here ("Extracted_data_analysis",                    "global
",                        "Output-Visualization"))
```

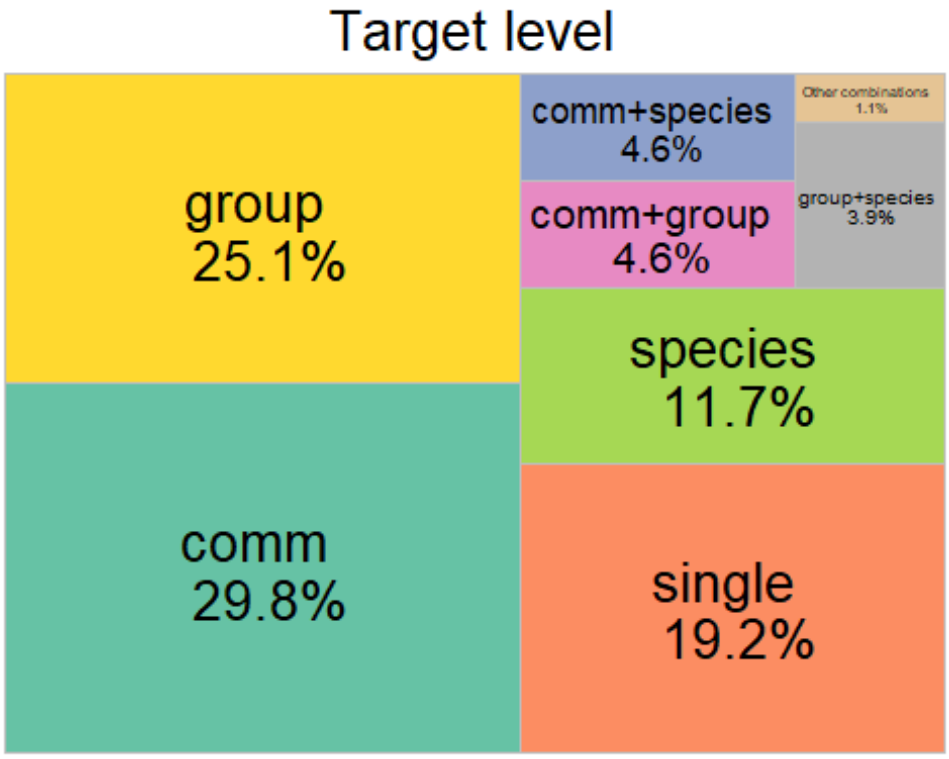
TARGET

Target level

```
target_level <- as.data.frame(count_results["taxonomicTargetLevel"])

# Rename columns
target_level %<>%
  rename(Level = taxonomicTargetLevel.taxonomicTargetLevel,
         Count = taxonomicTargetLevel.Count,
         Percentage = taxonomicTargetLevel.Percentage)

# Remove rows with nd
target_level <- target_level[!target_level$Level %in% "species,species",]
target_level <- target_level[!target_level$Level %in% ",species",]

# Merge habit combination with less than 3% and recalculate Count and Percent
ages
target_level <- target_level %>%
  mutate(Level = if_else(Percentage < 3, "Other combinations", Level)) %>%
  group_by(Level) %>%
  select(-Percentage) %>%
  summarise(
    Count = sum(Count)) %>%
  mutate(Percentage = round(Count / sum(Count) * 100,1)
  )
```

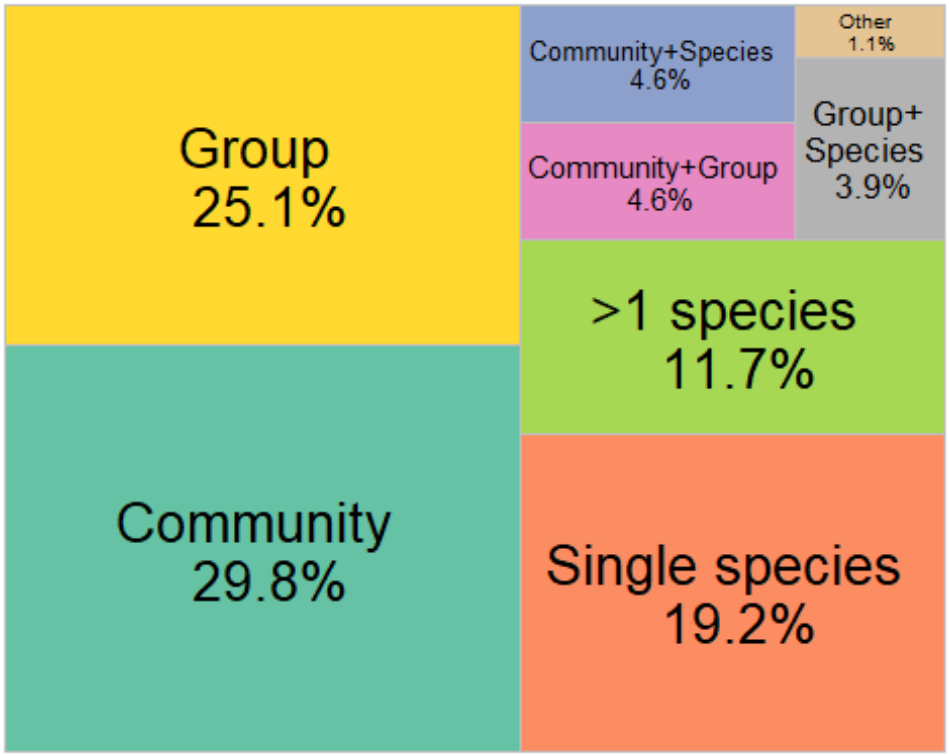
Treemap with the taxonomic target level

```
ggplot(target_level,
       aes(area=Count,
           fill=Level,
           line = "black",
           label=paste0(Level,
                        " \n ",Percentage,"%"))
       )+
  treemapify::geom_treemap(layout = "squarified")+
  treemapify::geom_treemap_text(place = "centre",
                    size = 20)+
  labs(title = "Target level")+
    theme(plot.title = element_text(hjust = 0.5, size= 20),
          legend.position = "none")+
  scale_fill_manual(values= c("#66C2A5","#E78AC3" , "#8DA0CB","#FFD92F","#B3B
3B3", "#E5C494","#FC8D62","#A6D854"))
```




```
target_level_labels <- target_level %<>%
  mutate(
    Level = case_when(Level == "comm" ~ "Community",
                      Level == "group"~ "Group",
                      Level == "single" ~ "Single species",
                      Level == "species" ~ ">1 species",
                      Level == "comm+group" ~ "Community+Group",
                      Level == "comm+species" ~ "Community+Species",
                      Level == "group+species" ~ "Group+\nSpecies",
                      Level == "Other combinations" ~ "Other")
    )

target_level_treemap <- ggplot(target_level_labels,
       aes(area=Count,
           fill=Level,
           line = "black",
           label=paste0(Level,
                        " \n ",Percentage,"%"))
       )+
  treemapify::geom_treemap(layout = "squarified")+
  treemapify::geom_treemap_text(place = "centre",
                    size = 20)+
    theme(plot.title = element_text(hjust = 0.5, size= 20),
          legend.position = "none")+
  scale_fill_manual(values= c("#A6D854" ,"#66C2A5","#E78AC3", "#8DA0CB","#FFD
92F","#B3B3B3", "#E5C494", "#FC8D62"))
```

```
target_level_treemap
```




```
ggsave("target_level_treemap.png",
       plot = target_level_treemap,
       units = "in", width = 9, height = 6,
       path = here ("Extracted_data_analysis",                    "global
",                        "Output-Visualization"))
```

Detected phyla in community studies

```
detected <- raw %>%
    filter(str_detect(taxonomicTargetLevel,"comm")) %>%
  select(File,
         detected_meso_cnidaria,
         detected_meso_sponges,
         detected_meso_seagrass,
         detected_meso_algae,
         detected_meso_mollusca,
         detected_meso_chondrichthyes,
         detected_meso_fish,
         detected_meso_crustacea,
         detected_meso_echino,
         detected_meso_other_taxa)

detected_lvls <-
  unique(
```

```
    unlist(
      str_split(detected$detected_meso_other_taxa, "\\+")
                                )
                           )
detected_lvls<- detected_lvls[! detected_lvls %in% c("",NA,"notapp",",","nd")]
detected[,detected_lvls] <- sapply(detected_lvls,
                                   function(y){
                                     str_count(detected$detected_meso_othe
r_taxa, y)
                                     }
                                   )

rm(detected_lvls)

detected %<>%
  mutate(across(everything(), as.character)) %>%
  replace(is.na(.), "0") %>%
  select(-detected_meso_other_taxa) %>%
  filter(if_all(everything(), ~ !grepl("notapp", .x)))

detected <- detected %>%
  mutate(across(-File, ~ case_when(
    .x %in% c("x", "x,", "xx") ~ "1",
    .x == "," ~ "0",
    TRUE ~ .x
  ))) %>%
  mutate(across(-File, as.numeric))
```

Calculate number of publications per taxonomic group

```
detected_tot <-
  detected %>%
  summarise(across(where(is.numeric),
                   list(sum)
                   )
            )

detected_tot  <- reshape2::melt(detected_tot, id.vars = NULL)

colnames(detected_tot) <- c("Taxa", "Count")

detected_tot %<>%
        mutate_at("Taxa",
                  stringr::str_replace, "_1", "") %>%
        mutate_at("Taxa",
                  stringr::str_replace, "detected_meso_", "")


detected_tot %<>%
```

```
      mutate(Percentage= round(Count / nrow(detected) * 100,1))

rm(detected)
```

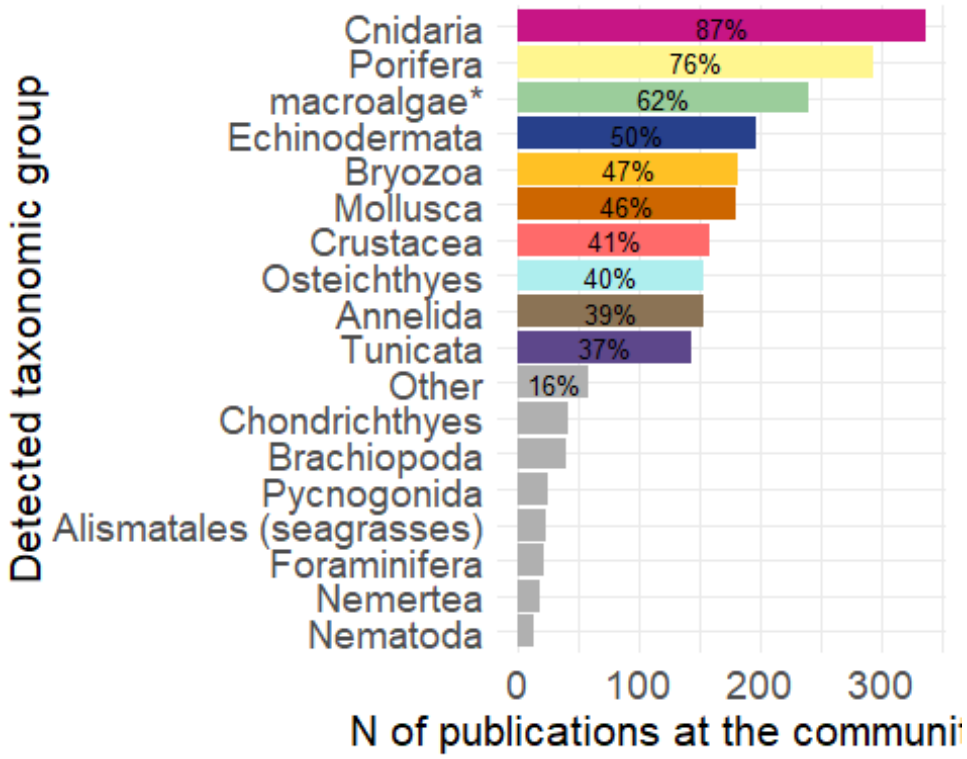
Merge poorly represented taxonomic groups into “Other”

```
detected_tot_collapsed <- detected_tot %>%
  mutate(Taxa = if_else(Percentage < 3, "Other", Taxa)) %>%
  group_by(Taxa) %>%
  summarise(
    Count = sum(Count),
    Percentage = sum(Percentage)
  )
```

Create a bar chart with detected taxonomic groups

```
detected_bar <- ggplot(detected_tot_collapsed, aes(x = reorder(Taxa, Count),
y = Count, fill= Taxa)) +
  geom_bar(stat = "identity") +
  geom_text(
    data = detected_tot_collapsed %>% filter(Percentage > 11),
    aes(label = paste0(round(Percentage, 0), "%")),
    position = position_stack(vjust = 0.5),
    color = "black",
    size = 3.5
  )+
  labs(y = "N of publications at the community level", x = "Detected taxonomi
c group") +
  theme_minimal() +
  coord_flip()+  # optional: flips for horizontal bars
  theme(legend.position ="none")+
  scale_x_discrete(labels = rev(c("Cnidaria",
                                  "Porifera",
                                  "macroalgae*",
                                  "Echinodermata",
                                  "Bryozoa",
                                  "Mollusca",
                                  "Crustacea",
                                  "Osteichthyes",
                                  "Annelida",
                                "Tunicata",
                                "Other",
                                "Chondrichthyes",
                                "Brachiopoda",
                                "Pycnogonida",
                                "Alismatales (seagrasses)",
                                "Foraminifera",
                                "Nemertea",
                                "Nematoda")))
```

```
detected_palette <- c("algae" ="darkseagreen3",
                      "cnidaria" = "mediumvioletred",
                      "crustacea" = "indianred1",
                      "sponges" = "khaki1",
                      "mollusca" = "darkorange3",
                      "fish" = "paleturquoise2",
                      "echino" = "royalblue4",
                      "Tunicata" = "mediumpurple4",
                      "chondrichthyes" = "gray69",
                      "Brachiopoda" = "gray69",
                      "Other" = "gray69",
                      "Nemertea" = "gray69",
                      "Pycnogonida" = "gray69",
                      "seagrass" =  "gray69",
                      "Nematoda" = "gray69",
                      "Foraminifera"   =    "gray69",
                      "Ciliata" = "gray69",
                      "Bryozoa" = "goldenrod1",
                      "Annelida" = "burlywood4")

detected_bar +
  scale_fill_manual(values = detected_palette,
    limits = names(detected_palette))+
  theme(axis.text = element_text(size = 14),
        axis.title = element_text(size = 16)
        )
```



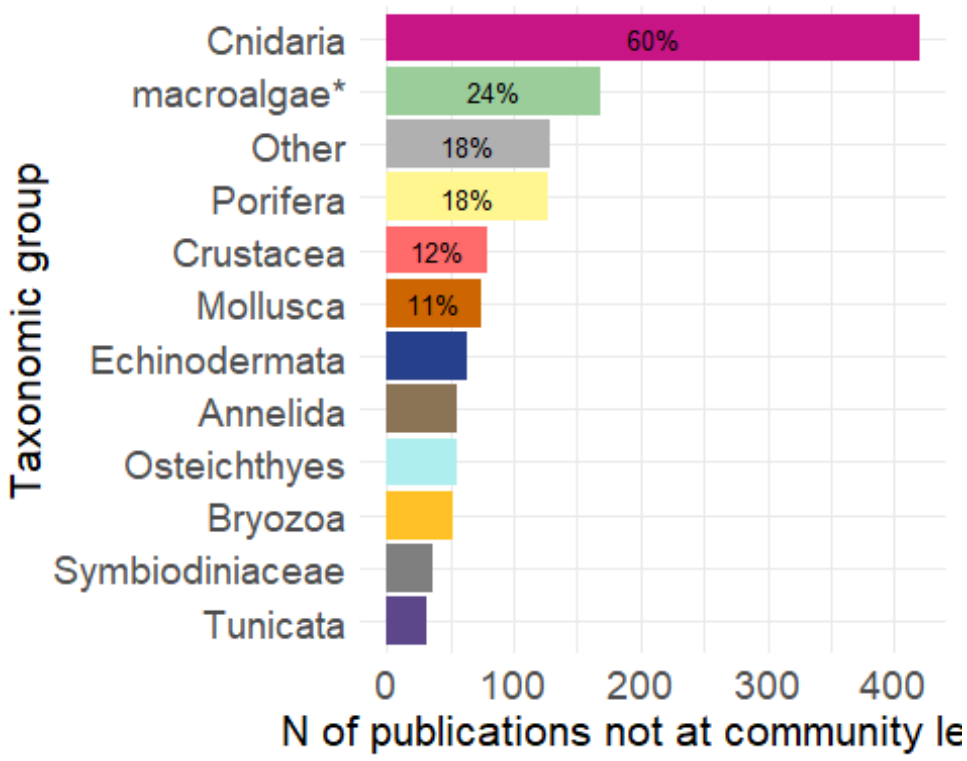
```
ggsave("detected_bar.png",
       units = "in", width = 7, height = 6,
       path = here ("Extracted_data_analysis",                          "global
",                          "Output-Visualization"))
```

Target taxonomic groups not at the community level

```
group <- raw %>%
    filter(taxonomicTargetLevel != "comm") %>%
  select(File,
         detected_meso_cnidaria,
         detected_meso_sponges,
         detected_meso_seagrass,
         detected_meso_algae,
         detected_meso_mollusca,
         detected_meso_chondrichthyes,
         detected_meso_fish,
         detected_meso_crustacea,
         detected_meso_echino,
         detected_meso_other_taxa,
         targetTaxa)

group_lvls <-
  unique(
    unlist(
      str_split(group$detected_meso_other_taxa, "\\+")
                                    )
                           )
group_lvls<- group_lvls[! group_lvls %in% c("",NA,"notapp",",","nd")]
group[,group_lvls] <- sapply(group_lvls,
                                        function(y){
                                          str_count(group$detected_meso_other_t
axa, y)
                                          }
                                        )

rm(group_lvls)

group <- group %>%
  mutate(across(everything(), as.character)) %>%
  replace(is.na(.), "0") %>%
  select(-detected_meso_other_taxa,-targetTaxa) %>%
  filter(if_all(everything(), ~ !grepl("notapp", .x))) %>%
  mutate(across(-File, ~ case_when(
    .x %in% c("x", "x,", "xx") ~ "1",
    .x == "," ~ "0",
    TRUE ~ .x
  ))) %>%
  mutate(across(-File, as.numeric))
```

```r
group_tot <-
  group %>%
  summarise(across(where(is.numeric),
                   list(sum)
                   )
            )

group_tot  <- reshape2::melt(group_tot, id.vars = NULL)

colnames(group_tot) <- c("Taxa", "Count")

group_tot %<>%
        mutate_at("Taxa",
                  stringr::str_replace, "_1", "") %>%
        mutate_at("Taxa",
                  stringr::str_replace, "detected_meso_", "")



group_tot %<>%
      mutate(Percentage= round(Count / nrow(group) * 100,1))

group_tot_collapsed <- group_tot %>%
  mutate(Taxa = if_else(Percentage < 3, "Other", Taxa)) %>%
  group_by(Taxa) %>%
  summarise(
    Count = sum(Count),
    Percentage = sum(Percentage),
    .groups = "drop"
  )

group_tot_collapsed %<>%
  mutate(Taxa = case_when(Taxa == 'echino' ~ "Echinodermata",
                          Taxa == 'fish' ~ "Osteichthyes",
                          Taxa == 'mollusca' ~ "Mollusca",
                          Taxa == 'seagrass' ~ "Alismatales (seagrasses)",
                          Taxa == "chondrichthyes" ~ "Chondrichthyes",
                          Taxa == "sponges" ~ "Porifera",
                          Taxa == "crustacea" ~ "Crustacea",
                          Taxa == "cnidaria" ~ "Cnidaria",
                          Taxa == "algae" ~ "macroalgae*",
         .default = Taxa))

group_bar <- ggplot(group_tot_collapsed, aes(x = reorder(Taxa, Count), y = Co
unt, fill= Taxa)) +
  geom_bar(stat = "identity") +
  geom_text(
    data = group_tot_collapsed %>% filter(Percentage > 10),
    aes(label = paste0(round(Percentage, 0), "%")),
    position = position_stack(vjust = 0.5),
    color = "black",
```

```
    size = 3.5
  )+
  labs(y = "N of publications not at community level", x = "Taxonomic group")
 +
  theme_minimal() +
  coord_flip()+  # optional: flips for horizontal bars
  theme(legend.position ="none")

group_palette <- c("macroalgae*" ="darkseagreen3",
                       "Cnidaria" = "mediumvioletred",
                       "Crustacea" = "indianred1",
                       "Porifera" = "khaki1",
                       "Mollusca" = "darkorange3",
                       "Osteichthyes" = "paleturquoise2",
                       "Echinodermata" = "royalblue4",
                       "Tunicata" = "mediumpurple4",
                       "Chondrichthyes" = "gray69",
                       "Other" = "gray69",
                       "Alismatales (seagrasses)" =  "gray69",
                       "Bryozoa" = "goldenrod1",
                       "Annelida" = "burlywood4")

group_bar <- group_bar +
  scale_fill_manual(values = group_palette,
    limits = names(group_palette))+
    theme(axis.text = element_text(size = 14),
        axis.title = element_text(size = 16)
        )

group_bar
```



Export bar plot

```
ggsave("group_bar.png",
       plot = group_bar,
       units = "in", width = 7, height = 6,
       path = here ("Extracted_data_analysis",                    "global
",                        "Output-Visualization"))
```

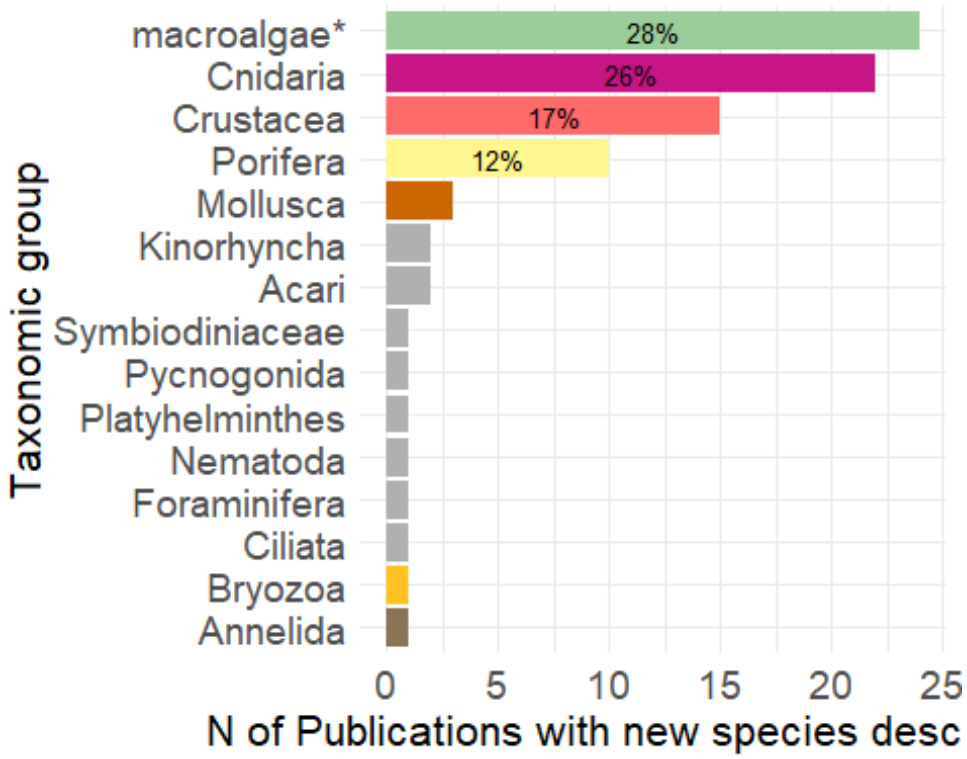
New species described per phyla

```
new_spec <- raw %>%
  filter(str_detect(taxonomic_publication, "taxa"))

new_spec %<>%
  select(File,
         detected_meso_cnidaria,
         detected_meso_sponges,
         detected_meso_seagrass,
         detected_meso_algae,
         detected_meso_mollusca,
         detected_meso_chondrichthyes,
         detected_meso_fish,
         detected_meso_crustacea,
         detected_meso_echino,
         detected_meso_other_taxa)

new_spec_lvls <-
  unique(
    unlist(
```

```
      str_split(new_spec$detected_meso_other_taxa, "\\+")
                                  )
                          )
new_spec_lvls<- new_spec_lvls[! new_spec_lvls %in% c("",NA,"notapp")]
new_spec[,new_spec_lvls] <- sapply(new_spec_lvls,
                                   function(y){
                                     str_count(new_spec$detected_meso_othe
r_taxa, y)
                                     }
                                   )

rm(new_spec_lvls)

new_spec  %<>%
  mutate(across(everything(), as.character)) %>%
  replace(is.na(.), "0") %>%
  select(-detected_meso_other_taxa) %>%
  filter(if_all(everything(), ~ !grepl("notapp", .x))) %>%
  mutate(across(-File, ~ case_when(
    .x %in% c("x") ~ "1",
    TRUE ~ .x
  ))) %>%
  mutate(across(-File, as.numeric))

new_spec_tot <-
  new_spec %>%
  summarise(across(where(is.numeric),
                   list(sum)
                   )
            )

rm(new_spec)
new_spec_tot  <- reshape2::melt(new_spec_tot, id.vars = NULL)

colnames(new_spec_tot) <- c("Taxa", "Count")

new_spec_tot %<>%
        mutate_at("Taxa",
                  stringr::str_replace, "_1", "") %>%
          mutate_at("Taxa",
                  stringr::str_replace, "detected_meso_", "")



new_spec_tot %<>%
  filter(Count>0) %>%
      mutate(Percentage= round(Count / sum(Count) * 100,1))

new_spec_plot <-   ggplot(new_spec_tot,
                          aes(x = reorder(Taxa, Count), y = Count,
                              fill= Taxa)) +
```

```r
  geom_bar(stat = "identity") +
  geom_text(
    data = new_spec_tot %>% filter(Percentage > 8),
    aes(label = paste0(round(Percentage, 0), "%")),
    position = position_stack(vjust = 0.5),
    color = "black",
    size = 3.5
  )+
  labs(y = "N of Publications with new species descriptions", x = "Taxonomic
group") +
  theme_minimal() +
  coord_flip()+  # optional: flips for horizontal bars
  theme(legend.position ="none")+
  scale_x_discrete(labels = rev(c("macroalgae*",
                                  "Cnidaria",
                                  "Crustacea",
                                  "Porifera",
                                  "Mollusca",
                                  "Kinorhyncha",
                                  "Acari",
                                  "Symbiodiniaceae",
                                "Pycnogonida",
                                "Platyhelminthes",
                                "Nematoda",
                                "Foraminifera",
                                "Ciliata",
                                "Bryozoa",
                                "Annelida"))
                   )

new_spec_palette <- c("algae" ="darkseagreen3",
                      "cnidaria" = "mediumvioletred",
                      "crustacea" = "indianred1",
                      "sponges" = "khaki1",
                      "mollusca" = "darkorange3",
                      "Kinorhyncha" = "gray69",
                      "Acari" = "gray69",
                      "Symbiodiniaceae" = "gray69",
                      "Pycnogonida" = "gray69",
                      "Platyhelminthes" =  "gray69",
                      "Nematoda" = "gray69",
                      "Foraminifera"   =    "gray69",
                      "Ciliata" = "gray69",
                      "Bryozoa" = "goldenrod1",
                      "Annelida" = "burlywood4")

new_species_bar <- new_spec_plot +
  scale_fill_manual(values = new_spec_palette,
    limits = names(new_spec_palette))+
    theme(axis.text = element_text(size = 14),
```

```
        axis.title = element_text(size = 16)
        )

new_species_bar
```



Export bar plot

```
ggsave("new_species_bar.png",
       plot = new_species_bar,
       units = "in", width = 7, height = 6,
       path = here ("Extracted_data_analysis",                    "global
",                        "Output-Visualization"))
```

New species descriptions that included molecular analyses

```
new_spec_molecular <- raw %>%
  filter(str_detect(taxonomic_publication, "taxa") &
           molecularAnalyses != "no" &
           molecularAnalyses != "nd" &
           molecularAnalyses != "notapp")

nrow(new_spec_molecular)

## [1] 35
```

DATA CROSSING

```
table(raw$meso_protection)

##
## mixed    nd    no   yes
##   108   656    72   165
```

VISUALLY SURVEYED SURFACE

```
raw$visuallySurveyedSurface <- as.factor(raw$visuallySurveyedSurface)
summary(subset(raw, is.na(raw$taxonomic_publication))$visuallySurveyedSurface)

##           <1           >1          >10         >100        >1000       >10
000
##           11           28           79          129          118
 63
##      >100000     >1000000    >10000000   >100000000 >10000000000
 nd
##           24            8            1            1            1
354
##           no       notapp
##            2          105
```

Platform per latitudinalRegion

```
nrow(subset(platform_depth_lat,platform_depth_lat$ccr_direct == 1))

## [1] 91

nrow(subset(platform_depth_lat,platform_depth_lat$ccr_direct == 1 & grepl("tr
opical",platform_depth_lat$latitudinalRegion)))

## [1] 80

nrow(subset(platform_depth_lat,platform_depth_lat$ccr_direct == 1 & grepl("te
mperate",platform_depth_lat$latitudinalRegion)))

## [1] 12

nrow(subset(platform_depth_lat,platform_depth_lat$hov == 1 & grepl("tropical",
platform_depth_lat$latitudinalRegion)))

## [1] 86

nrow(subset(platform_depth_lat,platform_depth_lat$hov == 1 & grepl("temperate
",platform_depth_lat$latitudinalRegion)))

## [1] 38

nrow(subset(platform_depth_lat,platform_depth_lat$hov == 1))

## [1] 128
```

```
nrow(subset(platform_depth_lat,platform_depth_lat$hov == 1 & grepl("polar",pl
atform_depth_lat$latitudinalRegion)))
```

```
## [1] 6
```

Observation

```
print(paste("Number of manipulative experimental studies:",nrow(subset(raw, g
repl("exp", raw$observationalORexperimental)))))
```

```
## [1] "Number of manipulative experimental studies: 98"
```

```
print(paste("Number of manipulative experimental studies in the field of rest
oration:", nrow(subset(raw, grepl("exp", raw$observationalORexperimental) & r
aw$field_rest != 0)

               )
            )
      )
```

```
## [1] "Number of manipulative experimental studies in the field of restorati
on: 6"
```

Temp replicates & exp

```
print(paste("# Publications that included temporal replicates:", nrow(subset
(raw, grepl("yes", temporalReplicates)))))
```

```
## [1] "# Publications that included temporal replicates: 244"
```

```
print(paste("# Publications that included temporal replicates:", nrow(subset
(raw, (grepl("exp", raw$observationalORexperimental) & grepl("yes", temporalR
eplicates))))))
```

```
## [1] "# Publications that included temporal replicates: 68"
```

Health

```
health <- subset(raw, grepl("Health", indicators) | grepl("disease", indicato
rs))

unique(health$targetTaxa)
```

```
##  [1] "Corallinales"

##  [2] "no"

##  [3] "Paramuricea_clavata"

##  [4] "Corallium_rubrum"

##  [5] "Bryozoa+Corallium_rubrum+Leptopsammia_pruvoti+Madracis_pharensis"
```

```
##  [6] "Eunicella_cavolini+Paramuricea_clavata"

##  [7] "Pachastrella monilifera+Poecillastra_compressa"

##  [8] "Peltaster_placenta+Peltaster_placenta"

##  [9] "Agaricia_grahamae+Agaricia_undata+Madracis_pharensis"

## [10] "Scleractinia"

## [11] "Eunicella_verrucosa"

## [12] "Corallium_rubrum+Eunicella_cavolini"

## [13] "Adeonella_calveti+Myriapora_truncata+Pentapora_fascialis+Reteporella
_grimaldii+Smittina_cervicornis"
## [14] "Eunicella_cavolini+Eunicella singularis+Paramuricea_clavata"

## [15] "Cidaris_spp+Corallium_rubrum+Eunicella_cavolinii"

## [16] "Orbicella_spp+Scleractinia"

## [17] "Anthipathella_subpinnata+Eunicella_cavolini+Paramuricea_clavata"

## [18] "Viminella_flagellum"

## [19] "Eunicella_singularis"

## [20] "Porifera"

## [21] "Octocorallia"

## [22] "Anthozoa"

## [23] "notapp"

## [24] "Alveopora_allingi+Alveopora_ocellata"

## [25] "Eunicella_cavolinii+Eunicella_singularis+Eunicella_verrucosa+Paramur
icea_clavata"
## [26] "Placopecten_magellanicus"

## [27] "Xestospongia_muta"

## [28] "Amphiprion_chrysopterus"
```

```
## [29] "Scleractinia+Sarcophyton_spp"

## [30] "Eunicella_cavolini+Eunicella_singularis+Paramuricea_clavata"

## [31] "Antipathes_dichotoma+Antipathes_grandis+Carijoa_riisei"

## [32] "Leptoseris_fragilis+Symbiodiniaceae"

## [33] "Pachyseris"

## [34] "Agaricia_lamarcki"

## [35] "Eunicella_singularis+Paramuricea_clavata"

## [36] "Balticina_willemoesi"

## [37] "Errina_novaezelandiae"

## [38] "Anthipatharia"

## [39] "Eunicella_cavolini"

## [40] "Montastraea_cavernosa+Siderastrea_stellata"

## [41] "Stylophora_pistillata+Symbiodiniaceae"

## [42] "Seriatopora_hystrix"

## [43] "Acropora_austera+Anthozoa"

## [44] "Ectocarpus_sp+Clodophora_sp+Lyngbya_sp+Pseudoanabaena_sp"

## [45] "Antipatharia+Octocorallia"

## [46] "Plakortis_angulospiculatus"

## [47] "Antillogorgia_bipinnata+Ellisella_elongata+Octocorallia+Porifera"

## [48] "Orbicella_spp"

## [49] "Montastraea_cavernosa+Symbiodiniaceae"

## [50] "Antipatharia+Octocorallia+Porifera"

## [51] "Spinimuricea_klavereni"

## [52] "Agaricia_lamarcki+Orbicella_franksi"
```

```
## [53] "Desmophyllum_pertusum"
```

ROV & sample

```
rov <- subset(raw, grepl("rov", meso_samplingPlatform) )

rov_specimen <- subset(rov,  grepl("specimen", collectedDataType) )

print(paste("Percentage of publications relying solely on ROV where specimens
 were collected:", round(nrow(rov_specimen)/nrow(rov)*100,1),"%"))
```

```
## [1] "Percentage of publications relying solely on ROV where specimens were
 collected: 44.8 %"
```

```
rm(rov, rov_specimen)
```

Molecular techniques in taxonomic and non-taxonomic studies

```
raw$taxonomic_publication <- as.factor(raw$taxonomic_publication )

molecular <- raw %>%
  filter(molecularAnalyses != "no" &
           molecularAnalyses != "notapp" &
           molecularAnalyses != "nd")

print(paste("# Publications that adopted molecular techniques: ",nrow(molecul
ar)))
```

```
## [1] "# Publications that adopted molecular techniques:  159"
```

```
molecular_notaxa <- molecular %>%
  filter(is.na(taxonomic_publication))

print(paste("# Publications that adopted molecular techniques not for taxonom
y: ",nrow(molecular_notaxa)))
```

```
## [1] "# Publications that adopted molecular techniques not for taxonomy:  1
24"
```

```
rm(molecular, molecular_notaxa)
```

Most common target species

```
target_species <- raw %>%
  count(targetTaxa, name = "Count") %>%
  arrange(desc(Count))

target_species
```

```
## # A tibble: 443 × 2
##    targetTaxa            Count
```

```
##    <chr>                  <int>
##  1 notapp                   170
##  2 no                       114
##  3 Scleractinia              45
##  4 Corallium_rubrum          26
##  5 Porifera                  20
##  6 Corallinales              13
##  7 Paramuricea_clavata       12
##  8 Octocorallia              11
##  9 Montastraea_cavernosa     10
## 10 Rhodophyta                10
## # ℹ 433 more rows
```

```
colnames(raw)
```

```
##  [1] "File"                          "recordTitle"
##  [3] "recordAuthors"                 "publicationYear"
##  [5] "province"                      "prov_code"
##  [7] "ecoreg"                        "scale"
##  [9] "ecoreg_code"                   "location"
## [11] "mininumSurveyedDepthInMeters" "maximumSurveyedDepthInMeters"
## [13] "latitudinalRegion"             "meso_direct_indirect"
## [15] "meso_samplingPlatform"         "taxonomic_publication"
## [17] "meso_substrate"                "meso_habitat"
## [19] "meso_protection"               "taxonomicTargetLevel"
## [21] "habit"                         "environmentalPosition"
## [23] "detected_meso_seagrass"        "detected_meso_algae"
## [25] "detected_meso_cnidaria"        "detected_meso_sponges"
## [27] "detected_meso_echino"          "detected_meso_mollusca"
## [29] "detected_meso_chondrichthyes" "detected_meso_fish"
## [31] "detected_meso_crustacea"       "detected_meso_other_taxa"
## [33] "indicators"                    "collectedDataType"
## [35] "meso_protocol_campaign"        "samplingUnit"
## [37] "visuallySurveyedSurface"       "meso_design"
## [39] "meso_replicates"               "temporalReplicates"
## [41] "abioticVariablesCollection"   "abioticVariables"
## [43] "molecularAnalyses"             "targetTaxa"
## [45] "DOI"                           "meso_dimensions"
## [47] "observationalORexperimental"  "max_depth_direct"
## [49] "max_depth_hov"                 "max_depth_rov"
## [51] "min_depth_hov"                 "min_depth_rov"
## [53] "max_depth_ccr"                 "max_depth_open"
## [55] "min_depth_ccr"                 "min_depth_open"
## [57] "max_depth_scuba"               "min_depth_scuba"
## [59] "field_cons"                    "field_commdesc"
## [61] "field_map"                     "field_impass"
## [63] "field_troph"                   "field_taxo"
## [65] "field_dist"                    "field_aqua"
## [67] "field_method"                  "field_physio"
## [69] "field_func"                    "field_habtype"
```

```
## [71] "field_fishery"                "field_stock"
## [73] "field_paleo"                  "field_phylo"
## [75] "field_rest"                   "field_archaeo"
## [77] "field_micro"                  "field_biomedicine"
## [79] "field_chem"                   "field_Molecular_ecology"
## [81] "field_Acoustics"              "field_environmental_drivers"
## [83] "field_Behaviour"              "field_Geomorphology"
## [85] "field_Taxonomy"
```

```
print(paste("Publications on Corallium rubrum :", nrow(subset(raw, grepl("rub
rum", targetTaxa)))))
```

```
## [1] "Publications on Corallium rubrum : 32"
```

```
print(paste("Publications on Paramuricea clavata:", nrow(subset(raw, grepl("c
lavata", targetTaxa)))))
```

```
## [1] "Publications on Paramuricea clavata: 27"
```

```
print(paste("Publications on Montastraea cavernosa:", nrow(subset(raw, grepl
("cavernosa", targetTaxa)))))
```

```
## [1] "Publications on Montastraea cavernosa: 27"
```

SESSION INFORMATION AND REFERENCES

```
## R version 4.5.2 (2025-10-31 ucrt)
## Platform: x86_64-w64-mingw32/x64
## Running under: Windows 11 x64 (build 26200)
##
## Matrix products: default
##   LAPACK version 3.12.1
##
## locale:
## [1] LC_COLLATE=Italian_Italy.utf8  LC_CTYPE=Italian_Italy.utf8
## [3] LC_MONETARY=Italian_Italy.utf8 LC_NUMERIC=C
## [5] LC_TIME=Italian_Italy.utf8
##
## time zone: Europe/Rome
## tzcode source: internal
##
## attached base packages:
## [1] stats     graphics  grDevices utils     datasets  methods   base
##
## other attached packages:
##  [1] RColorBrewer_1.1-3 treemapify_2.6.0   openxlsx_4.2.8.1   tibble_3.3.1

##  [5] purrr_1.2.1        stringr_1.6.0      magrittr_2.0.4     readxl_1.4.5

##  [9] ggplot2_4.0.2      dplyr_1.2.0        here_1.0.2
```

```
##
## loaded via a namespace (and not attached):
##  [1] utf8_1.2.6        generics_0.1.4    tidyr_1.3.2       stringi_1.8.7

##  [5] digest_0.6.39     evaluate_1.0.5    grid_4.5.2        fastmap_1.2.0

##  [9] cellranger_1.1.0  rprojroot_2.1.1   plyr_1.8.9        ggrepel_0.9.6

## [13] zip_2.3.3         scales_1.4.0      textshaping_1.0.4 cli_3.6.5

## [17] rlang_1.1.7       withr_3.0.2       yaml_2.3.12       otel_0.2.0

## [21] tools_4.5.2       reshape2_1.4.5    forcats_1.0.1     vctrs_0.7.1

## [25] R6_2.6.1          lifecycle_1.0.5   ragg_1.5.0        pkgconfig_2.0.3

## [29] pillar_1.11.1     gtable_0.3.6      glue_1.8.0        Rcpp_1.1.1

## [33] ggfittext_0.10.3  systemfonts_1.3.1 xfun_0.56         tidyselect_1.2.
1
## [37] rstudioapi_0.18.0 knitr_1.51        farver_2.1.2      htmltools_0.5.9

## [41] rmarkdown_2.30    svglite_2.2.2     labeling_0.4.3    compiler_4.5.2

## [45] S7_0.2.1
```

```
library(report)
report::report(sessionInfo())
```

```
## Analyses were conducted using the R Statistical language (version 4.5.2; R
 Core
## Team, 2025) on Windows 11 x64 (build 26200), using the packages magrittr
## (version 2.0.4; Bache S, Wickham H, 2025), report (version 0.6.3; Makowski
 D et
## al., 2023), here (version 1.0.2; Müller K, 2025), tibble (version 3.3.1; M
üller
## K, Wickham H, 2026), RColorBrewer (version 1.1.3; Neuwirth E, 2022), openx
lsx
## (version 4.2.8.1; Schauberger P, Walker A, 2025), ggplot2 (version 4.0.2;
## Wickham H, 2016), stringr (version 1.6.0; Wickham H, 2025), readxl (versio
n
## 1.4.5; Wickham H, Bryan J, 2025), dplyr (version 1.2.0; Wickham H et al.,
## 2026), purrr (version 1.2.1; Wickham H, Henry L, 2026) and treemapify (ver
sion
## 2.6.0; Wilkins D, 2025).
##
## References
## ----------
##   - Bache S, Wickham H (2025). _magrittr: A Forward-Pipe Operator for R_.
```

```
## doi:10.32614/CRAN.package.magrittr
## <https://doi.org/10.32614/CRAN.package.magrittr>, R package version 2.0.4,
## <https://CRAN.R-project.org/package=magrittr>.
##   - Makowski D, Lüdecke D, Patil I, Thériault R, Ben-Shachar M, Wiernik B
(2023).
## "Automated Results Reporting as a Practical Tool to Improve Reproducibilit
y and
## Methodological Best Practices Adoption." _CRAN_.
## doi:10.32614/CRAN.package.report
## <https://doi.org/10.32614/CRAN.package.report>,
## <https://easystats.github.io/report/>.
##   - Müller K (2025). _here: A Simpler Way to Find Your Files_.
## doi:10.32614/CRAN.package.here <https://doi.org/10.32614/CRAN.package.her
e>, R
## package version 1.0.2, <https://CRAN.R-project.org/package=here>.
##   - Müller K, Wickham H (2026). _tibble: Simple Data Frames_.
## doi:10.32614/CRAN.package.tibble
## <https://doi.org/10.32614/CRAN.package.tibble>, R package version 3.3.1,
## <https://CRAN.R-project.org/package=tibble>.
##   - Neuwirth E (2022). _RColorBrewer: ColorBrewer Palettes_.
## doi:10.32614/CRAN.package.RColorBrewer
## <https://doi.org/10.32614/CRAN.package.RColorBrewer>, R package version 1.
1-3,
## <https://CRAN.R-project.org/package=RColorBrewer>.
##   - R Core Team (2025). _R: A Language and Environment for Statistical
## Computing_. R Foundation for Statistical Computing, Vienna, Austria.
## <https://www.R-project.org/>.
##   - Schauberger P, Walker A (2025). _openxlsx: Read, Write and Edit xlsx F
iles_.
## doi:10.32614/CRAN.package.openxlsx
## <https://doi.org/10.32614/CRAN.package.openxlsx>, R package version 4.2.8.
1,
## <https://CRAN.R-project.org/package=openxlsx>.
##   - Wickham H (2016). _ggplot2: Elegant Graphics for Data Analysis_.
## Springer-Verlag New York. ISBN 978-3-319-24277-4,
## <https://ggplot2.tidyverse.org>.
##   - Wickham H (2025). _stringr: Simple, Consistent Wrappers for Common Str
ing
## Operations_. doi:10.32614/CRAN.package.stringr
## <https://doi.org/10.32614/CRAN.package.stringr>, R package version 1.6.0,
## <https://CRAN.R-project.org/package=stringr>.
##   - Wickham H, Bryan J (2025). _readxl: Read Excel Files_.
## doi:10.32614/CRAN.package.readxl
## <https://doi.org/10.32614/CRAN.package.readxl>, R package version 1.4.5,
## <https://CRAN.R-project.org/package=readxl>.
##   - Wickham H, François R, Henry L, Müller K, Vaughan D (2026). _dplyr: A
Grammar
## of Data Manipulation_. doi:10.32614/CRAN.package.dplyr
## <https://doi.org/10.32614/CRAN.package.dplyr>, R package version 1.2.0,
## <https://CRAN.R-project.org/package=dplyr>.
```

```
##   - Wickham H, Henry L (2026). _purrr: Functional Programming Tools_.
## doi:10.32614/CRAN.package.purrr <https://doi.org/10.32614/CRAN.package.pur
rr>,
## R package version 1.2.1, <https://CRAN.R-project.org/package=purrr>.
##   - Wilkins D (2025). _treemapify: Draw Treemaps in 'ggplot2'_.
## doi:10.32614/CRAN.package.treemapify
## <https://doi.org/10.32614/CRAN.package.treemapify>, R package version 2.6.
0,
## <https://CRAN.R-project.org/package=treemapify>.
```